Digital Logic and Computer Design

Unit1

Combinational and Sequential Logic Design

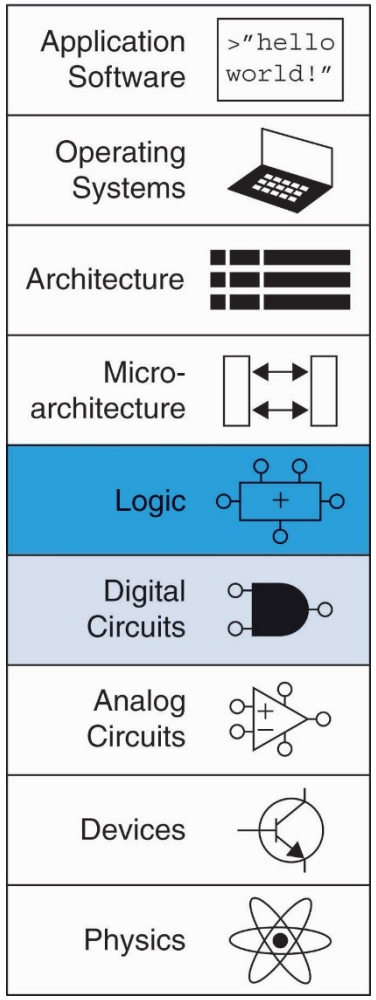
Topic:Introduction

Team DLCD
Department of CSE and CSE(AI &ML)

UNIT 1

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

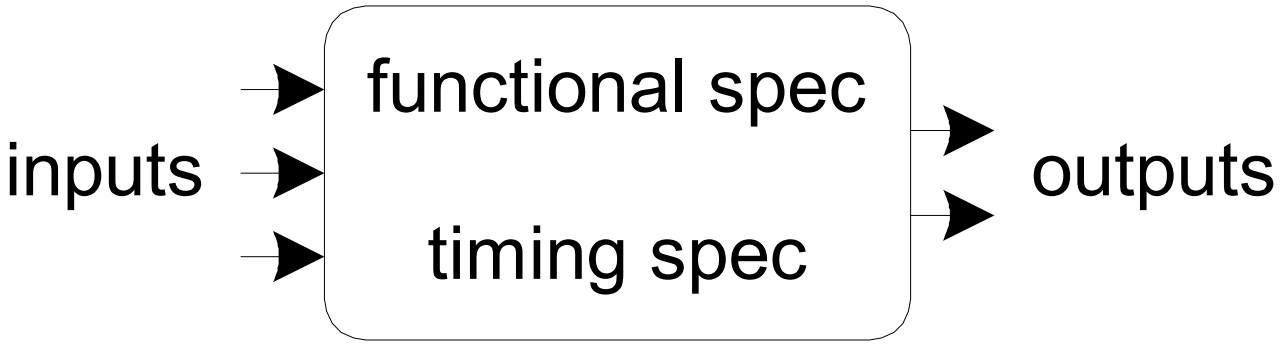
- Introduction
- Boolean Equations
- Karnaugh maps
- Combinational building Blocks.
- Introduction
- Latches and Flip-Flops
- Synchronous Logic Design
- Finite state Machines



Introduction

What is a circuit?

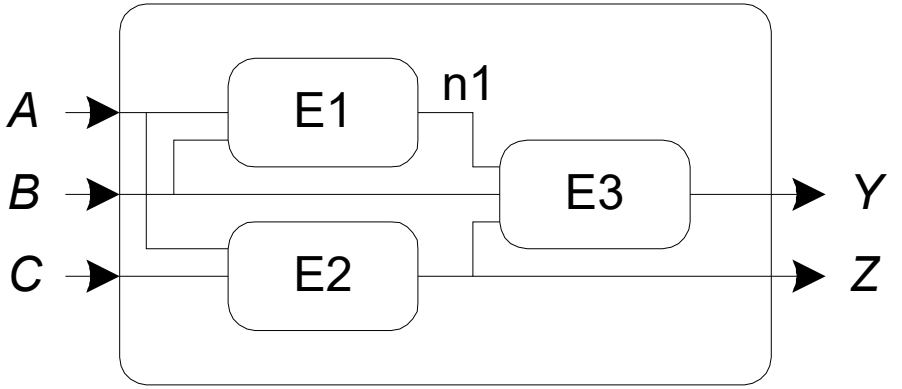
- In digital electronics, a circuit is a network that processes discrete-valued variables.
- A circuit can be viewed as a black box, shown as in Figure with
  - one or more discrete-valued input terminals
  - one or more discrete-valued output terminals
  - a functional specification describing the relationship between inputs and outputs
  - a timing specification describing the delay between inputs changing and outputs responding.

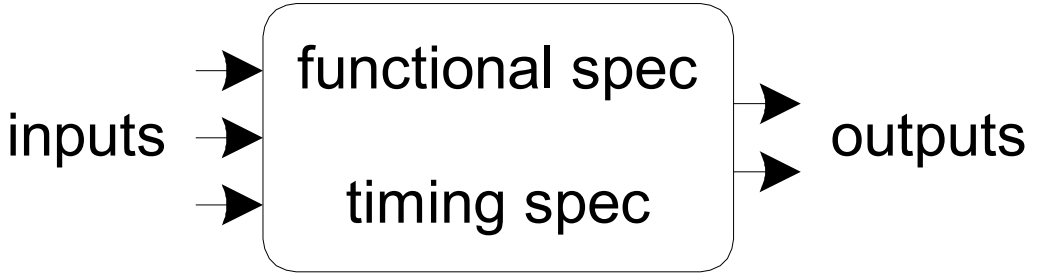


COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Introduction

- **Nodes**
  - Inputs: *A*, *B*, *C*
  - Outputs: *Y*, *Z*
  - Internal: n1
- **Circuit elements**
  - E1, E2, E3
  - Each a circuit



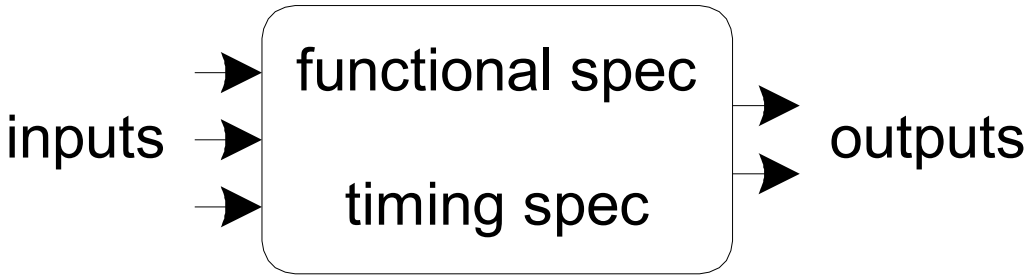


COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Introduction

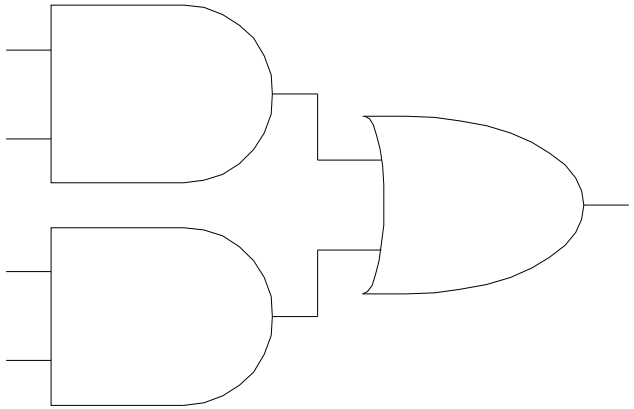
Types of Digital circuits/logic

- **Combinational Logic**
  - Memory less
  - Outputs determined only by current values of inputs
- Examples:
- Adders, Multiplexers, Decoders, Encoders, Comparators



- **Sequential Logic**
  - Has memory
  - Outputs determined by previous and current values of inputs
- Examples:
- Registers, Counters, Flip-flops, Memory devices

Introduction



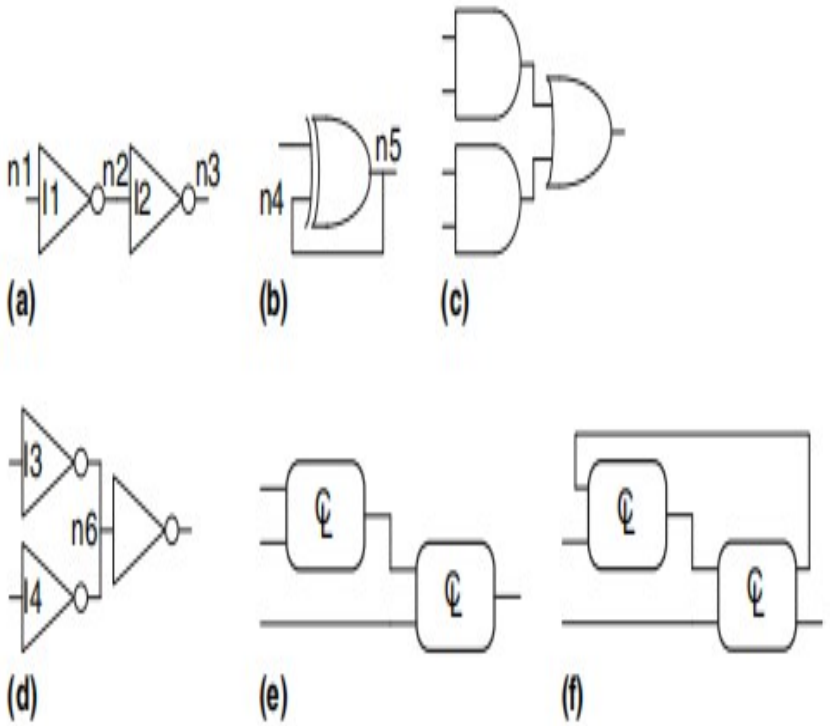
Combinational Logic

- A circuit is combinational if it consists of interconnected circuit elements such that
  - Every circuit element is itself combinational.
  - Every node of the circuit is either designated as an input to the circuit or connects to exactly one output terminal of a circuit element.
  - The circuit contains no cyclic paths: every path through the circuit visits each circuit node at most once.

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Introduction

Which of the circuits are combinational circuits according to the rules of combinational composition?



Circuit (a) is combinational.

(b) is not combinational, because there is a cyclic path starting at n4 passes through the XOR to n5,which returns to n4.

(c) is combinational.

(d) is not combinational, because node n6 connects to the output terminals of both I3 and I4.

(e)Is combinational, illustrating two combinational circuits connected to form a larger combinational circuit.

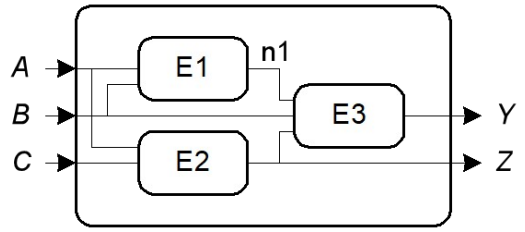
(f) does not obey the rules of combinational composition because it has a cyclic path through the two elements. Depending on the functions of the elements, it may or may not be a combinational circuit

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Think About it?

MCQ 1

A designer replaces E2 with a faster implementation that preserves the same Boolean function.
Which statement is most accurate?



A. Functional specification changes.
B. Timing specification may improve while functionality remains unchanged.
C. Both functional and timing specifications change.
D. Overall logic becomes sequential.

Correct Answer B

Think About it?

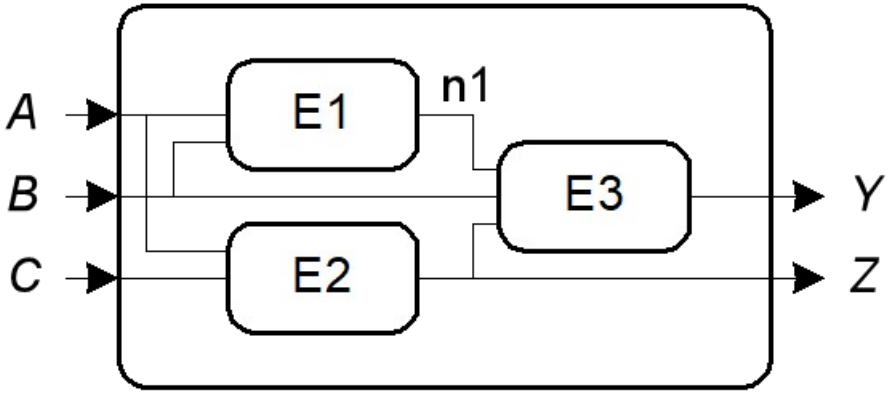
MCQ 2

Assume:

Delay(E1) = 5 ns

Delay(E2) = 8 ns

Delay(E3) = 4 ns



E3 begins computation only after both E1 and E2 outputs become valid.

What is the worst-case propagation delay?

A. 9 ns

B. 12 ns

Correct Answer: B

Explanation:

Longest input path = max(5,8)+4 = 12 ns

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Think About it?

MCQ 3

Consider two digital circuits:

Circuit X: Output depends only on current inputs.

Circuit Y: Output depends on previous and current inputs.

Which classification is correct?

A. X is Sequential, Y is Combinational

B. X is Combinational, Y is Sequential

C. Both are Sequential

D. Both are Combinational

Correct Answer B
X is Combinational, Y is Sequential



THANK YOU

Team DLCD

Department of CSE and CSE(AI &ML)



Digital Logic and Computer Design

Unit1
Combinational and Sequential Logic Design
Topic:Boolean Equations

Team DLCD
Department of CSE and CSE(AI &ML)

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

Course Outline

Digital Design

Combinational logic design

Logic Minimization, K-Maps

Sequential logic design

Concepts covered

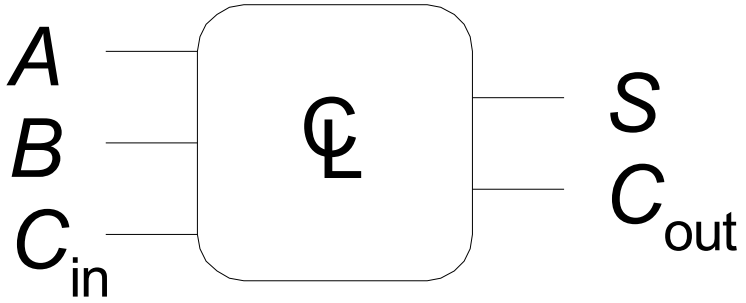
- Boolean Formula Terminology
- Sum of Products and Product of Sums standard forms
- Truth Table to Boolean Formula Logic Circuit
- Logic Minimization using Boolean Identities

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

- Functional specification of outputs in terms of inputs
- **Example:** $S = F(A, B, C_{in})$

$$C_{out} = F(A, B, C_{in})$$



$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

Boolean Equations

Terminology

- Complement: variable with a bar over it
  A', B', C'
- Literal: variable or its complement
  A, A', B, B', C, C'
- Implicant: product of literals
  ABC, A'C, BC'
- Minterm: product that includes all input variables
  A'B'C', AB'C,ABC
- Maxterm: sum that includes all input variables
  (A+B+C'), (A+B'+C), (A'+B'+C')

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Formula Terminology

Literal Boolean variable or its complement (ex:c')

Product (or implicant) AND of two or more literals (ex: *abc'*)

Minterm Product involving all inputs to Boolean function

a	*b*	*c*	minterm	name
0	0	0	a'b'c'	m_0
0	0	1	a'b'c	m_1
0	1	0	a'bc'	m_2
0	1	1	a'bc	m_3
1	0	0	ab'c'	m_4
1	0	1	ab'c	m_5
1	1	0	abc'	m_6

Minterms for three inputs *a*, *b* and *c*

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Formula Terminology

Sum OR of two or more literals

Maxterm Sum involving all inputs to Boolean function

a	*b*	*c*	maxterm	Name
0	0	0	a+b+c	M_0
0	0	1	a+b+c’	M_1
0	1	0	a+b’+c	M_2
0	1	1	a+b’+c’	M_3
1	0	0	a’+b+c	M_4
1	0	1	a’+b+c’	M_5
1	1	0	a’+b’+c	M_6
1	1	1	a’+b’+c’	

Maxterms for three inputs *a*, *b* and *c*

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

From Truth Table to Boolean Formula and its Minimization

Given a combinational logic circuit or Boolean formula, we have learnt to construct its truth table

But, given a truth table, how to construct a Boolean formula (or combinational logic circuit) for it?

Also, as there are multiple Boolean formulas / logic circuits for each truth table, how to pick the minimal one?

Above problem is called **logic minimization**

many metrics: smallest, fastest, least power consumption

our metric: smallest two level Sum of Products formula

may be more than one solution

Boolean Equations

Sum of Products Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	*B*	*Y*	minterm	minterm name
0	0	0	$\overline{A}\ \overline{B}$	m_0
0	1	1	$\overline{A}\ B$	m_1
1	0	0	$A\ \overline{B}$	m_2
1	1	1	$A\ B$	m_3

$Y = F(A, B) =$

Sum of Products Form

Boolean Equations

Sum of Products Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	*B*	*Y*	minterm	minterm name
0	0	0	$\overline{A}\,\overline{B}$	m_0
0	1	1	$\overline{A}\,B$	m_1
1	0	0	$A\,\overline{B}$	m_2
1	1	1	$A\,B$	m_3

$Y = F(A, B) =$

Sum-of-Products Form

Boolean Equations

Sum of Products Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	*B*	*Y*	minterm	minterm name
0	0	0	$\overline{A}\ \overline{B}$	m_0
0	1	1	$\overline{A}\ B$	m_1
1	0	0	$A\ \overline{B}$	m_2
1	1	1	$A\ B$	m_3

$$Y = F(A, B) = A'B + AB = \Sigma(1, 3)$$

Boolean Equations

Sum of Products Form

Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants.Design a circuit that will output TRUE only if Ben enjoys the picnic

Solution:First define the inputs and outputs. The inputs are A and R, which indicate if there are ants and if it rains. A is TRUE when there are ants and FALSEwhen there are no ants.

A	R	E
0	0	1
0	1	0
1	0	0
1	1	0

Figure 2.10 Ben's truth table

Likewise, R is TRUE when it rains and FALSE when the sun smiles on Ben. The output is E, Ben's enjoyment of the picnic. E is TRUE

if Ben enjoys the picnic and FALSE if he suffers.

Using sum-of-products form, we write the equation as: E = A'R' or

E = Σ0

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN
Boolean Formula using Sum of Products (SOP)

SOP Form

Sum of all minterms corresponding to a 1 output

SOP Example

Truth table:

a	*b*	*c*	*y*	minterm	name
0	0	0	0	a'b'c'	m0
0	0	1	0	a'b'c	m1
0	1	0	0	a'bc'	m2
0	1	1	1	a'bc	m3
1	0	0	1	ab'c'	m4
1	0	1	0	ab'c	m5
1	1	0	1	abc'	m6
1	1	1	1	*abc*	m7

SOP form Boolean formula:

$$y = a'bc + ab'c' + abc' + abc$$

$$y = m3 + m4 + m6 + m7$$

Sigma (Σ) notation:

$$y = \Sigma(m3, m4, m6, m7)$$

$$y = \Sigma(3, 4, 6, 7)$$

If the Boolean function is called *f* :

$$y = f(a, b, c)$$

Boolean Equations

Product of Sums Form

- All Boolean equations can be written in POS form
- Each row has a **maxterm**
- A maxterm is a sum (OR) of literals
- Each maxterm is FALSE for that row (and only that row)
- Form function by ANDing the maxterms for which the output is FALSE
- Thus, a product (AND) of sums (OR terms)

$$Y = F(A, B) = (A + B)(A' + B) = \Pi(0, 2)$$

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN
Boolean Formula using Products of Sum (POS)

POS Form

Product of all maxterms corresponding to a 0 output

POS Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Formula using Products of Sum (POS)

POS Form

Product of all maxterms corresponding to a 0 output

POS Example

Truth table:

a	*b*	*c*	*y*	maxterm	name
0	0	0	0	a+b+c	M_0
0	0	1	0	a+b+c’	M_1
0	1	0	0	a+b’+c	M_2
0	1	1	1	a+b’+c’	M_3
1	0	0	1	a’+b+c	M_4
1	0	1	0	a’+b+c’	M_5
1	1	0	1	a’+b’+c	

SOP form Boolean formula:

$y = (a + b + c)(a + b + c')$

$(a + b' + c)(a' + b + c')$

$y = M_0 M_1 M_2 M_5$

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Formula using Products of Sum (POS)

POS Form

Product of all maxterms corresponding to a 0 output

POS Example

Truth table:

a	*b*	*c*	*y*	maxterm	name
0	0	0	0	a+b+c	M_0
0	0	1	0	a+b+c’	M_1
0	1	0	0	a+b’+c	M_2
0	1	1	1	a+b’+c’	M_3
1	0	0	1	a’+b+c	M_4
1	0	1	0	a’+b+c’	M_5
1	1	0	1	a’+b’+c	

SOP form Boolean formula:

$y = (a + b + c)(a + b + c')(a + b' + c)(a' + b + c')$

$y = M_0 M_1 M_2 M_5$

Pi (Π) notation:

$y = \Pi(M_0 M_1 M_2 M_5)$

$y = \Pi(0, 1, 2, 5)$

Boolean Formula using Products of Sum (POS)

POS Form

Product of all maxterms corresponding to a 0 output

POS Example

Truth table:

a	*b*	*c*	*y*	maxterm	name
0	0	0	0	a+b+c	M_0
0	0	1	0	a+b+c’	M_1
0	1	0	0	a+b’+c	M_2
0	1	1	1	a+b’+c’	M_3
1	0	0	1	a’+b+c	M_4
1	0	1	0	a’+b+c’	M_5
1	1	0	0	a’+b’+c	

If the Boolean function is called f :

$y = f(a, b, c)$

SOP form Boolean formula:

$y = (a + b + c)(a + b + c')(a + b' + c)(a' + b + c')$

$y = M_0 M_1 M_2 M_5$

Pi (Π) notation:

$y = \Pi(M_0 M_1 M_2 M_5)$

$y = \Pi(0, 1, 2, 5)$

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

- You are going to the cafeteria for lunch
  - You won't eat lunch (E)
  - If it's not open (O) or
  - If they only serve corndogs (C)
- Write a truth table for determining if you will eat lunch (E).

O	*C*	*E*
0	0	
0	1	
1	0	
1	1	

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

- You are going to the cafeteria for lunch
  - You won't eat lunch (E)
  - If it's not open (O) or
  - If they only serve corndogs (C)
- Write a truth table for determining if you will eat lunch (E).

O	*C*	*E*
0	0	0
0	1	0
1	0	1
1	1	0

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

- SOP – sum-of-products

O	C	E	minterm
0	0	0	$\overline{O}\ \overline{C}$
0	1	0	$\overline{O}\ C$
1	0	1	$O\ \overline{C}$
1	1	0	$O\ C$

SOP & POS Form

$$E = O\overline{C}$$
$$= \Sigma(2)$$

- POS – product-of-sums

O	C	E	maxterm
0	0	0	$O + C$
0	1	0	$O + \overline{C}$
1	0	1	$\overline{O} + C$
1	1	0	$\overline{O} + \overline{C}$

$$E = (O + C)(O + \overline{C})(\overline{O} + \overline{C})$$
$$= \Pi(0, 1, 3)$$

Boolean Equations

SOP & POS Form

O	C	E	minterm
0	0		$\overline{O}\,\overline{C}$
0	1		$\overline{O}\,C$
1	0		$O\,\overline{C}$
1	1		$O\,C$

- SOP – sum-of-products

O	C	E	maxterm
0	0		$O + C$
0	1		$O + \overline{C}$
1	0		$\overline{O} + C$
1	1		$\overline{O} + \overline{C}$

- POS – product-of-sums

Logic Minimization

Logic Minimization

Example

Minimize:

$$y = a'bc + ab'c' + abc' + abc$$

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN
Logic Minimization

Logic Minimization Example

Minimize:

$y = a'bc + ab'c' + abc' + abc$
$y = bc(a' + a) + ac'(b' + b)$

Using distributive law:

Using complement law:

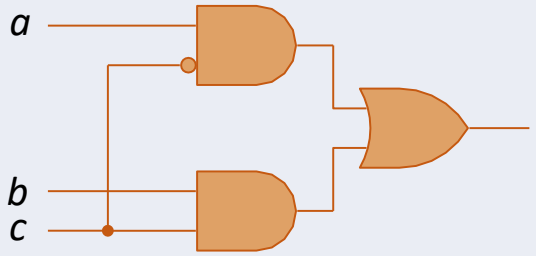
$y = bc(1) + ac'(1)$

Using identity law:

$y = bc + ac'$

The three representations below are different ways of specifying **the same Boolean function:**

$ac' + b\,c$



a	b	c	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Logic Minimization

Logic Minimization Example

Minimize:

$y = a'bc + abc' + abc' + abc$

Using distributive law:

$y = bc(1) + ac'(1)$

Using complement law:

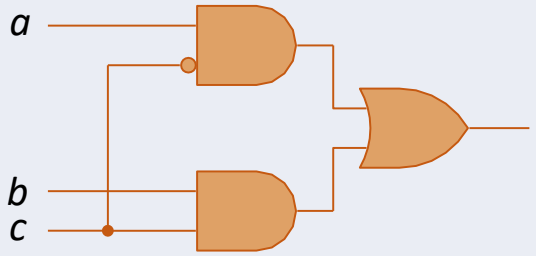
$y = bc(a' + a) + ac'(b' + b)$

Using identity law:

$y = bc + ac'$

The three representations below are different ways of specifying **the same Boolean function:**

$a \cdot \overline{c} + b \cdot c$



a	b	c	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Boolean Equations

MCQ 1

1.What is a " minterm" in Boolean algebra?

A) A sum of two or more literals
B) A product (AND) involving all inputs to the Boolean function
C) Any single literal
D) A product involving only some inputs

Correct Answer B

MCQ 2

2. In Product-of-Sums (POS) form, how is a Boolean formula constructed?

A) Sum of all minterms corresponding to a 1 output
B) Product of all maxterms corresponding to a 0 output
C) Product of all minterms corresponding to a 1 output
D) Sum of all maxterms corresponding to a 0 output

Correct Answer B

Boolean Equations

MCQ 3

A Boolean function has four inputs and is TRUE only for m(3,7,11,15)

Which characteristic do these minterms share?

A. A=1

B. D=1

C. C=0

D. B=1

Correct Answer B
(Each binary representation ends with D = 1.)

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Boolean Equations

Logic Circuit Example

Draw the logic circuit diagram for:

$y = \bar{a}bc + a\bar{b}\bar{c} + ab\bar{c} + abc$

Equivalence Example

Prove that: $bc + a\bar{c} + ab = bc + a\bar{c}$

Hint: $bc + a\bar{c} + ab(c + \bar{c})$

Minimization Example

Minimize:

$y = (a + b + c)(a + b + \bar{c})(a + b + c)(\bar{a} + b + \bar{c})$

Hint:

$y = (a(a + b + \bar{c}) + b(a + b + \bar{c}) + c(a + b + \bar{c}))$

$(a(\bar{a} + b + \bar{c}) + b(\bar{a} + b + \bar{c}) + c(\bar{a} + b + \bar{c}))$



THANK YOU

Team DLCD

Department of CSE and CSE(AI &ML)



DIGITAL LOGIC AND COMPUTER DESIGN

Karnaugh Maps

Team DLCD
Department of CSE and CSE(AI &ML)

DIGITAL LOGIC AND COMPUTER DESIGN

Karnaugh Maps

Department of CSE and CSE(AI &ML)

Karnaugh Maps- Three Input

Why use K-Maps?

a	*b*	*c*	*y*	minterm	name
0	0	0	0	a'b'c'	m_0
0	0	1	0	a'b'c	m_1
0	1	0	0	a'bc'	m_2
0	1	1	1	a'bc	m_3
1	0	0	1	ab'c'	m_4
1	0	1	0	ab'c	m_5
1	1	0	1	abc'	m_6
1	1	1	1	*abc*	m_7

Example 1

SOP form:

$y = a'bc + ab'c' + abc' + abc$

Minimized form:

$y = bc + ac'$

Minimization requires:

- Combining implicants on row 3 and 7
- Combining implicants on row 4 and 6
- Takes some effort to determine implicants to combine

Karnaugh Maps- Three input

Why use K-Maps?

a	b	c	y	minterm	name
0	0	0	0	a'b'c'	m_0
0	0	1	0	a'b'c	m_1
0	1	0	0	a'bc'	m_2
0	1	1	1	a'bc	m_3
1	0	0	1	ab'c'	m_4
1	0	1	0	ab'c	m_5
1	1	0	1	abc'	m_6
1	1	1	1	abc	m_7

Example 1

SOP form:

$y = a'bc + ab'c' + abc' + abc$

Minimized form:

$y = bc + ac'$

Minimization requires:

Combining implicants on row 3 and 7

Combining implicants on row 4 and 6

Takes some effort to determine implicants to combine

Minimizing:

$y = bc + ac' + ab$

Requires converting to:

$bc + ac' + ab(c + c')$

Minimization may require:

Unintuitve steps

Trial and error

So minimization with Boolean identities is important to know since useful in some cases, but difficult to use in general

Karnaugh Maps- Three input

K-Map Structure

Kanaugh Map (K-Map)

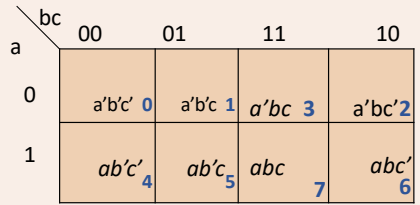
Maurice Karnaugh's refinement in 1953 of original idea from 1881

Key idea: Minterms that differ in one literal must be adjacent

Utilize brain's visual processing for efficient logic minimization

a	*b*	*c*	*y*	minterm
0	0	0	0	a'b'c'
0	0	1	0	a'b'c
0	1	0	0	a'bc'
0	1	1	1	a'bc
1	0	0	1	ab'c'
1	0	1	0	ab'c
1	1	0	1	abc'
1	1	1	1	*abc*

K-Map Structure (three input Boolean functon)



Each square corresponds to a row of the truth table

Any two adjacent squares differ only in one literal

Achieved using two rows and binary order 00,01,11,10

Notion of "wrap-around": far left and right squares are adjacent

Karnaugh Maps- Three input

K-Map Structure

Kanaugh Map (K-Map)

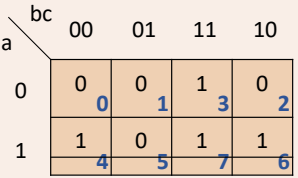
Maurice Karnaugh's refinement in 1953 of original idea from 1881

Key idea: Minterms that differ in one literal must be adjacent

Utilize brain's visual processing for efficient logic minimization

a	*b*	*c*	*y*	minterm
0	0	0	0	a'b'c'
0	0	1	0	a'b'c
0	1	0	0	a'bc'
0	1	1	1	a'bc
1	0	0	1	ab'c'
1	0	1	0	ab'c
1	1	0	1	abc'
1	1	1	1	*abc*

K-Map Structure (three input Boolean functon)



Each square corresponds to a row of the truth table

Any two adjacent squares differ only in one literal

Achieved using two rows and binary order 00,01,11,10

Notion of "wrap-around": far left and right squares are adjacent

Karnaugh Maps- Three input

K-Map Method

K-Map Implicants

- Implicant
  - K-Map area composed of squares containing 1's
  - Area is square or rectangular (wraparound allowed)
  - No. of squares in area is a power of two (1, 2, 4, . . .)
    - Each implicant corresponds to a product of literals
      - Double the area, one less literal
- Prime implicant
  - Implicant having largest number of squares obeying above rules
- Essential prime implicant
  - Prime implicant containing a square not in any other prime implicant

K-Map Method

- Include all required prime implicants
  - Include all essential prime implicants
  - Include other prime implicants such that:
    - Each square containing 1 is covered
    - Boolean formula is minimal (may not be unique)
- Convert required implicants to Boolean formula
  - Each implicant is a product of literals
  - Include literals which do not change over its area

Karnaugh Maps- Three input

MCQs

1. Why are K-Map rows/columns arranged in the order 00, 01, 11, 10 rather than 00, 01, 10, 11?

A) To make the K-Map easier to draw

B) So that any two adjacent squares differ in exactly one literal, matching the key K-Map principle

C) Because it matches decimal counting order

D) It's arbitrary, with no particular reason

Correct Answer
Option B

Karnaugh Maps- Three input

MCQs

2. What does "wrap-around" mean in a K-Map?

A) The far left and right (and, for larger maps, top and bottom) squares are considered adjacent

B) The K-Map can be rotated 90 degrees

C) Squares in different rows are never adjacent

D) Wrap-around only applies when using POS form

Correct Answer
Option A

Karnaugh Maps- Three input

MCQs

3. What defines a "prime implicant" in K-Map minimization?

A) Any implicant containing exactly one square

B) An implicant that is the largest possible grouping of adjacent 1s (can't be enlarged further while following the rules)

C) An implicant found only using Boolean algebra

D) An implicant covering all squares in the K-Map

Correct Answer
Option B

Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour

a \ bc	00	01	11	10
0	0 (0)	0 (1)	1 (3)	0 (2)
1	1 (4)	0 (5)	1 (7)	1 (6)

a	b	c	y	minterm
0	0	0	0	$\bar{a}\bar{b}\bar{c}$
0	0	1	0	$\bar{a}\bar{b}c$
0	1	0	0	$\bar{a}b\bar{c}$
0	1	1	1	$\bar{a}bc$
1	0	0	1	$a\bar{b}\bar{c}$
1	0	1	0	$a\bar{b}c$
1	1	0	1	$ab\bar{c}$
1	1	1	1	abc

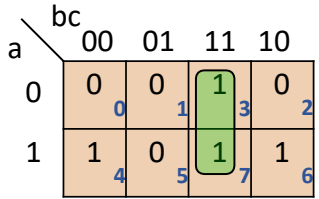
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



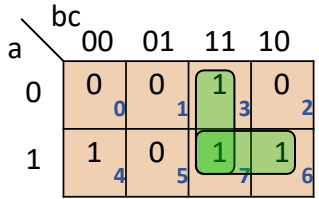
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Karnaugh Maps- Three input

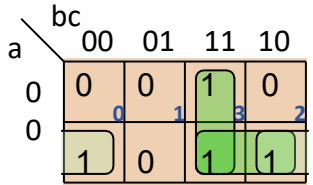
K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour

Three prime implicants



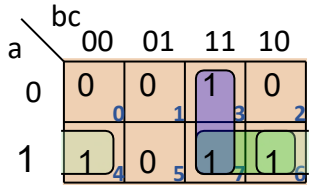
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Three prime implicants

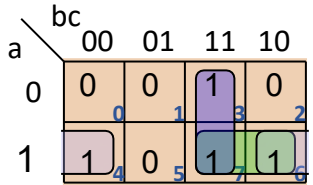
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Three prime implicants

Two essential prime implicants

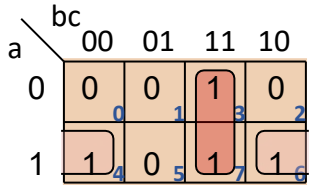
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Three prime implicants

Two essential prime implicants

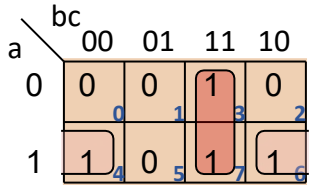
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Three prime implicants

Two essential prime implicants

Required prime implicants: *bc*,ac'

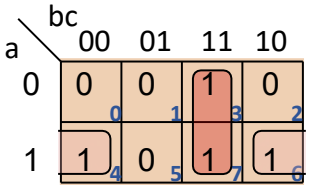
Karnaugh Maps- Three input

K-Map Example 1

Prime implicants: green colour

Essential prime implicants: blue colour

Required prime implicants: dark red colour



Three prime implicants

Two essential prime implicants

Required prime implicants: bc, ac'

Boolean formula:

$$f(a, b, c) = ac' + bc$$

Karnaugh Maps- Three input

K-Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

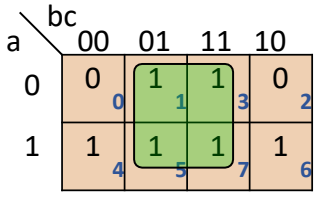
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



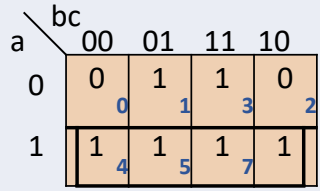
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



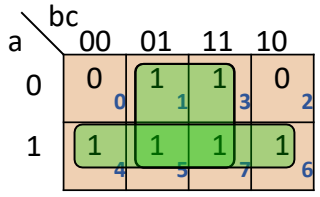
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants

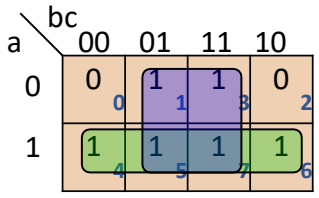
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants

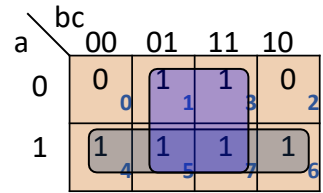
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants

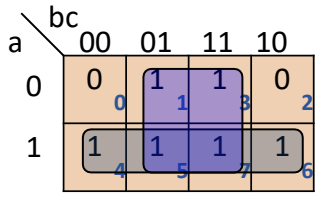
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants
Two essential prime implicants

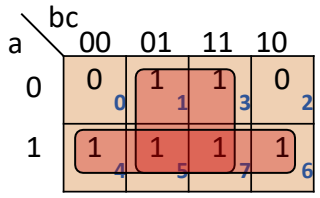
Karnaugh Maps- Three input

K Map Example 2

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants

Two essential prime implicants

Two required prime implicants: *c*, *a*

Karnaugh Maps- Three input

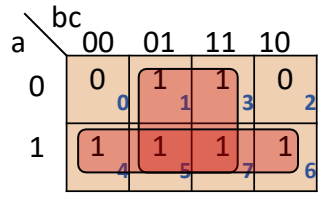
K Map Example 2

K-Map Minimization

Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Two prime implicants

Two essential prime implicants

Two required prime implicants: c, a

Minimal Boolean formula:

$f(a, b, c) = c + a$

Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

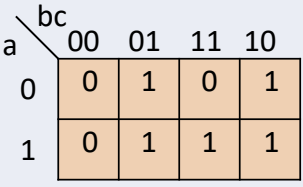
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Karnaugh Maps- Three input

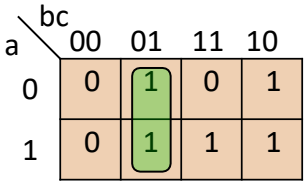
K Map Example 3

K-Map Minimization

Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Karnaugh Maps- Three input

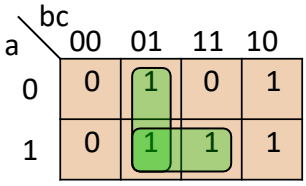
K Map Example 3

K-Map Minimization

Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



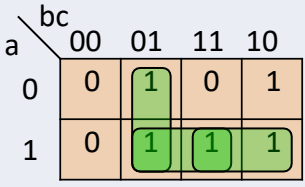
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



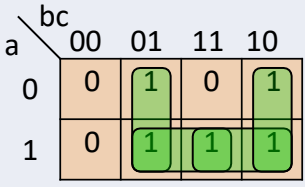
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



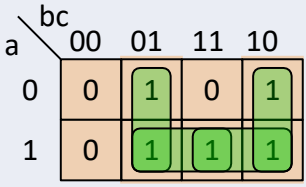
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Four prime implicants

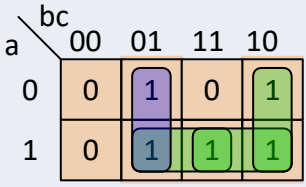
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Four prime implicants

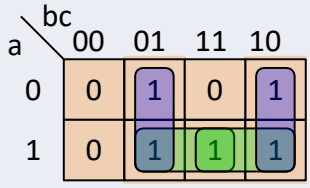
Karnaugh Maps- Three input

K Map Example 3

K-Map Minimization Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Four prime implicants

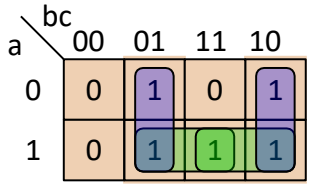
Karnaugh Three input
K Map Example 3

K-Map Minimization

Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Four prime implicants

Two essential prime implicants

Karnaugh Maps- Three input

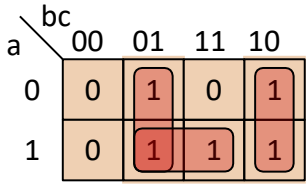
K Map Example 3

K-Map Minimization

Example

a	*b*	*c*	*y*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Four prime implicants

Two essential prime implicants

Three required prime implicants: b'c, b*c'* along with either ca or ab

Minimal Boolean formulas:

f (a, b, c) = b'c+ bc'+ ab

f (a, b, c) = b'c+ bc'+ac

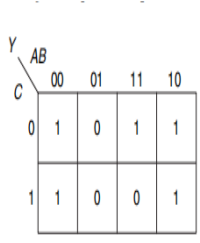
Karnaugh Maps- Three input

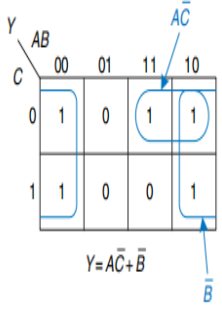
K Map Example 4

K-Map Minimization

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Truth table





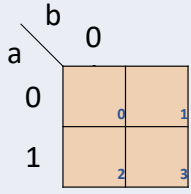
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



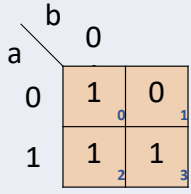
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



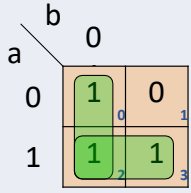
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



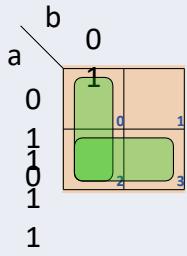
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

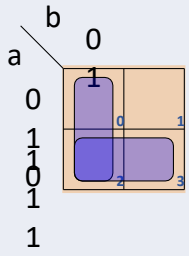
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

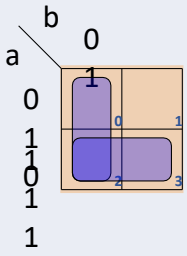
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

Two essential prime implicants

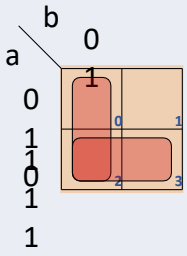
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

Two essential prime implicants

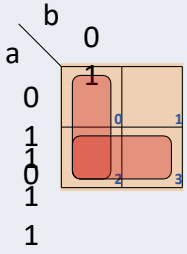
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	*b*	*y*
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

Two essential prime implicants

Two required prime implicants

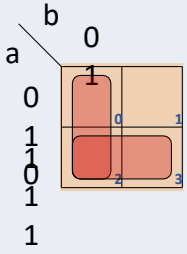
Karnaugh Maps-Two input

K-Map Example 1

K-Map Example (two inputs)

a	b	y
0	0	1
0	1	0
1	0	1
1	1	1

Two Input Truth Table



Two prime implicants

Two essential prime implicants

Two required prime implicants

Minimized Boolean formula:

$f(a, b, c, d) = \bar{b} + a$

Karnaugh Maps- Three input

Think About It

Minimize using K-Maps

1. $f(a, b, c) = \Sigma(0, 3, 5)$
2. $f(a, b, c) = \Sigma(0, 1, 2, 4, 5, 6)$
3. $f(a, b, c) = \Sigma(0, 2, 3, 4, 6, 7)$

All prime implicants and essential prime implicants shown for clarity, okay to directly mark required prime implicants on K-Map

Karnaugh Maps- Three input

MCQs

1. In a worked example with three prime implicants where two are essential, what determines which prime implicants are included in the final formula?

A) All prime implicants must always be included regardless of overlap

B) The essential prime implicants must be included (to cover their unique squares), plus any additional prime implicants needed to cover remaining 1s

C) Only the prime implicant with the fewest literals is used

D) Required prime implicants are chosen randomly

Correct Answer
Option B

Karnaugh Maps- Three input

MCQs

2. One example shows the minimal formula can be written as either f = bc' + b'c + ca or f = bc' + b'c + ab. What does this illustrate?

A) K-Maps always produce a unique minimal solution

B) The minimal SOP solution isn't always unique - different valid choices of non-essential prime implicants can give equally minimal formulas

C) POS form is always required in such cases

D) The K-Map method fails when there are multiple prime implicants

Correct Answer
Option B

Karnaugh Maps- Three input

MCQs

3. Regarding marking prime implicants, essential prime implicants, and required prime implicants on a K-Map, what does the slide note?

A) All three must always be marked with different colors in every solution

B) It's acceptable to skip marking all prime implicants and essential prime implicants separately, and directly mark just the required prime implicants

C) Only maxterms need to be marked

D) None of them need to be marked; only the final formula matters

Correct Answer
Option B

Karnaugh Maps-Four Inputs

K-Map Structure (four inputs)

K-Map Structure (four input Boolean function)

a	b	c	d	y	Minterm
0	0	0	0	1	a'b'c'd'
0	0	0	1	0	a'b'c'd
0	0	1	0	1	a'b'cd'
0	0	1	1	0	a'b'cd
0	1	0	0	0	a'bc'd'
0	1	0	1	1	a'bc'd
0	1	1	0	0	a'bcd'
0	1	1	1	1	a'bcd
1	0	0	0	1	ab'c'd'
1	0	0	1	0	ab'c'd
1	0	1	0	1	ab'cd'
1	0	1	1	0	ab'cd
1	1	0	0	0	abc'd'
1	1	0	1	1	abc'd
1	1	1	0	0	abcd'
1	1	1	1	1	abcd

Four Input Truth Table

ab \ cd	00	01	11	10
00	a'b'c'd' 0	a'b'c'd 1	a'b'cd 3	a'b'cd' 2
01	a'bc'd' 4	a'bc'd 5	a'bcd 7	a'bcd' 6
11	abc'd' 12	abc'd 13	abcd 15	abcd' 14
10	ab'c'd' 8	ab'c'd 9	ab'cd 11	ab'cd' 10

- Each square corresponds to a row of the truth table
- Any two adjacent squares differ only in one literal
- Columns as well as rows arranged in binary order 00,01,11,10
- Notion of "wrap-around" Far left and right squares are adjacent
- Top and bottom squares are also adjacent

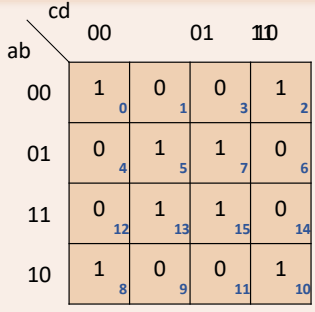
Karnaugh Maps-Four Inputs

K-Map Structure (four inputs)

K-Map Structure (four input Boolean function)

a	b	c	d	y	Minterm
0	0	0	0	1	a'b'c'd'
0	0	0	1	0	a'b'c'd
0	0	1	0	1	a'b'cd'
0	0	1	1	0	a'b'cd
0	1	0	0	0	a'bc'd'
0	1	0	1	1	a'bc'd
0	1	1	0	0	a'bcd'
0	1	1	1	1	a'bcd
1	0	0	0	1	ab'c'd'
1	0	0	1	0	ab'c'd
1	0	1	0	1	ab'cd'
1	0	1	1	0	ab'cd
1	1	0	0	0	abc'd'
1	1	0	1	1	abc'd
1	1	1	0	0	abcd'
1	1	1	1	1	abcd

Four Input Truth Table



Each square corresponds to a row of the truth table

Any two adjacent squares differ only in one literal

Columns as well as rows arranged in binary order 00,01,11,10

Notion of "wrap-around"

Far left and right squares are adjacent

Top and bottom squares are also adjacent

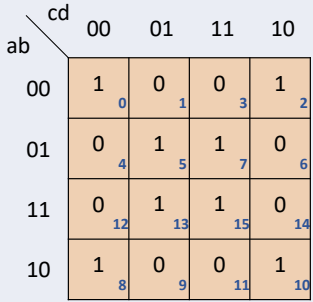
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



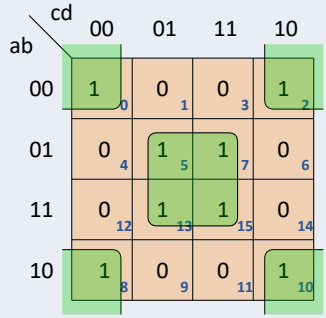
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



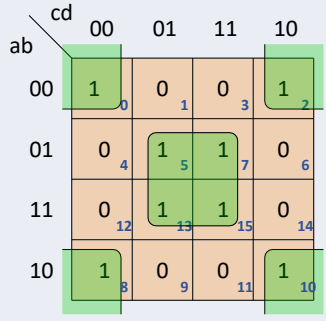
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

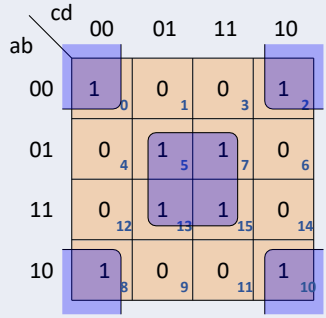
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

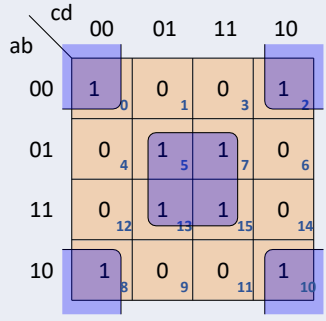
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

Two essential prime implicants

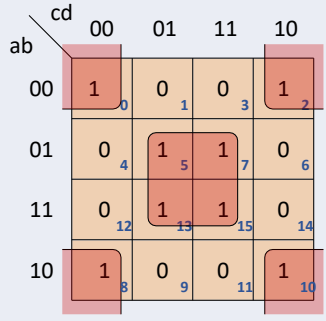
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

Two essential prime implicants

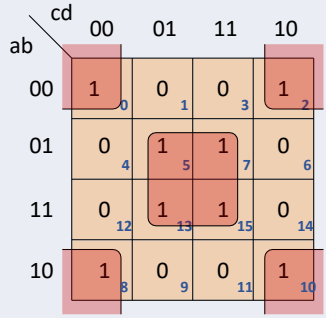
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

Two essential prime implicants

Two required prime implicants: *bd* , b'd'

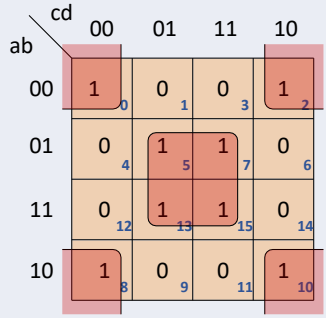
Karnaugh Maps-Four Inputs

K-Map Example 1 (four inputs)

K-Map Example (four inputs)

a	b	c	d	y
0	0	0	0	1
0	0	0	0	0
0	0	1	0	1
0	0	1	0	0
0	1	0	0	0
0	1	0	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	1	1
1	0	0	1	0
1	0	1	0	1
1	0	1	0	0
1	1	0	1	0
1	1	0	1	1
1	1	1	1	0
1	1	1	1	1

Four Input Truth Table



Two prime implicants

Two essential prime implicants

Two required prime implicants: bd , $b'd'$

Minimized Boolean formula:

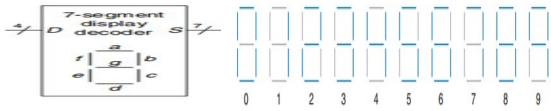
$f(a, b, c, d) = bd + b'd'$

Karnaugh Maps-Four Inputs

K-Map Example 2

A seven-segment display decoder takes a 4-bit data input D3:0 and produces seven outputs to control light-emitting diodes to display a digit from 0 to 9. The seven outputs are often called segments a through g, or Sa–Sg, as defined in Figure. The digits are shown.

Write a truth table for the outputs, and use K-maps to find Boolean equations for outputs Sa and Sb. Assume that illegal input values (10–15) produce a blank readout.

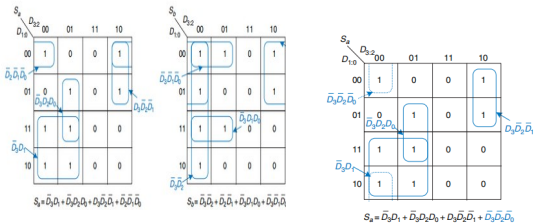


$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
others	0	0	0	0	0	0	0

Karnaugh Maps-Four Inputs

K-Map Example 2





Sa = D3' D1+ D3' D2D0 + D2'D1' D0' + D3 D2' D1'

Sb = D3' D2' + D2' D1' + D3' D1 D0 + D3' D1' D0'

Sc = D3' D0 + D3' D2 + D2' D1'

Sd = D3' D2' D1 + D3' D2 D1' D0+D2'D1'D0'+D3'D1D0

Se = D3' D1 D0' + D2' D1' D0'

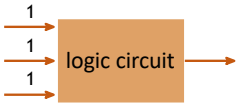
Sf = D3' D2 D0' + D3' D2 D1' + D3 D2' D1' + D3' D1' D0'

Sg = D3' D1 D0' + D3' D2' D1 + D3' D2 D1' + D3 D2' D1'

Karnaugh Maps-(Don't Cares)

Example 1(Three inputs)

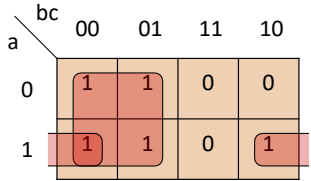
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	*b*	*c*	*y*
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

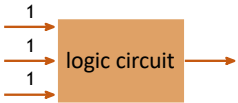
Truth table



Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

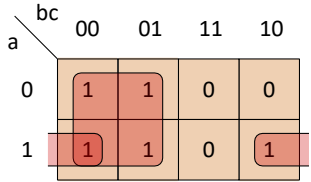
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Truth table

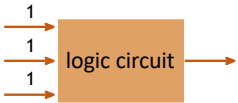


Case 0: $y = \bar{b} + \bar{c}a$

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

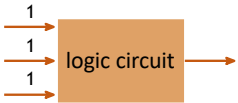
a \ bc	00	01	11	10
0	1	1	0	0
1	1	1	1	1

Case 0: $y = \bar{b} + \bar{c}a$

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

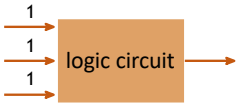
a \ bc	00	01	11	10
0	1	1	0	0
1	1	1	1	1

Case 0: $y = \bar{b} + \bar{c}a$

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

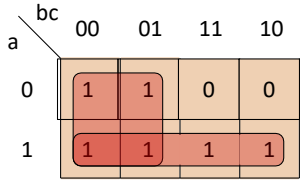
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	*b*	*c*	*y*
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table

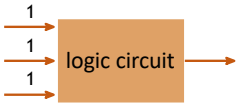


Case 1: $y = b' + a$

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

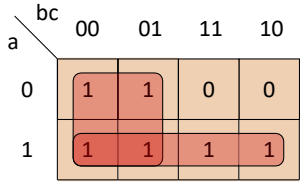
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



Case 0: $y = \bar{b} + \bar{c}\,a$

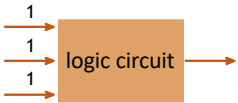
Case 1: $y = \bar{b} + a$

Either 0 or 1 can result in smaller formula

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

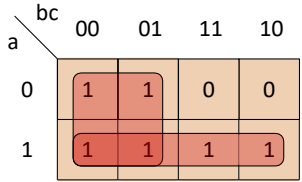
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Truth table



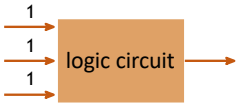
Case 0: $y = \bar{b} + \bar{c}a$

Case 1: $y = \bar{b} + a$

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

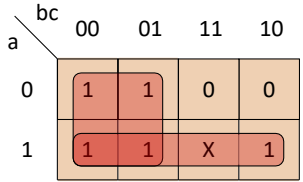
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	X

Truth table



Case 0: $y = \bar{b} + \bar{c}a$

Case 1: $y = \bar{b} + a$

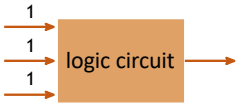
Either 0 or 1 can result in smaller formula

So initially write X in truth table and K-Map

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

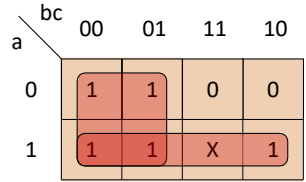
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	X

Truth table



Case 0: $y = \bar{b} + \bar{c}a$

Case 1: $y = \bar{b} + a$

Either 0 or 1 can result in smaller formula

So initially write X in truth table and K-Map

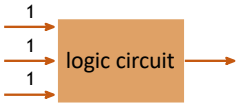
Don't Care

Output(s) we **don't care** about are denoted by X , can be treated as either 0 or 1

Karnaugh Maps-(Don't Cares)

Example 1 (Three inputs)

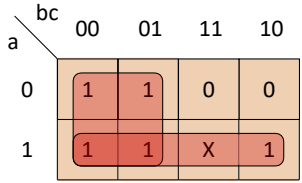
Suppose we are asked to design a three input logic circuit all whose inputs can never be 1 simultaneously



guaranteed can't happen

a	b	c	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	X

Truth table



Case 0: $y = \bar{b} + \bar{c}a$

Case 1: $y = \bar{b} + a$

Either 0 or 1 can result in smaller formula

So initially write X in truth table and K-Map

Don't Care

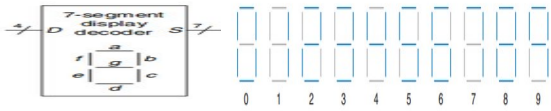
Output(s) we **don't care** about are denoted by X , can be treated as either 0 or 1

Karnaugh Maps-(Don't Cares)
Example 2(Four inputs)

K-Map Structure (four input Boolean function)

A seven-segment display decoder takes a 4-bit data input D3:0 and produces seven outputs to control light-emitting diodes to display a digit from 0 to 9. The seven outputs are often called segments a through g, or Sa–Sg, as defined in Figure. The digits are shown.

Write a truth table for the outputs, and use K-maps to find Boolean equations for outputs Sa and Sb. Use don't care about the output values for illegal input values of 10 to 15



S_a $D_{3:2}$ / $D_{1:0}$	00	01	11	10
00	1	0	X	1
01	0	1	X	1
11	1	1	X	X
10	1	1	X	X

$S_a = D_3 + D_2D_0 + \bar{D}_2\bar{D}_0 + D_1$

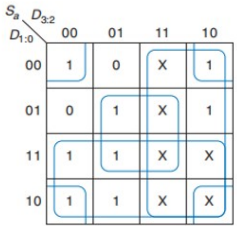
S_b $D_{3:2}$ / $D_{1:0}$	00	01	11	10
00	1	1	X	1
01	1	0	X	1
11	1	1	X	X
10	1	0	X	X

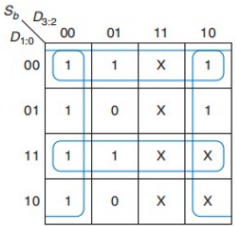
$S_b = \bar{D}_2 + D_1D_0 + \bar{D}_1\bar{D}_0$

Karnaugh Maps-Four Inputs

K-Map Example 2







Sa = D3+D2D0+D2’D0’+D1

Sb = D2' + D1'D0' + D1D0

Sc = D0 + D2 + D1’

Sd = D1D0' + D2’D1+ D0'D2' + D2D1’D0

Se = D1D0' + D2'D0’

Sf = D3 + D2D0' + D2D1' + D1’ D0'

Sg = D3 + D1D0' + D2'D1 + D2D1'

Karnaugh Maps-(Don't Cares)

Think About It

Minimize using K-Maps

K-Maps

1.f (a, b) = Σ(0, 1, 2, 3)

2.f (a, b, c, d) =Σ(0, 1, 5, 7, 15, 14, 10)

Minimize using

a	*b*	*c*	*d*	*y*
0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	1	0	0
0	1	0	0	0
0	1	0	0	0
0	1	1	1	0
0	1	1	1	0
1	0	0	1	1
1	0	0	1	1
1	0	1	0	1
1	0	1	0	*X*
1	1	0	1	0
1	1	0	1	0
1	1	1	1	0
1	1	1	1	*X*

Karnaugh Maps-(Don't Cares)

MCQs

1. In a four-input K-Map, which pairs of squares are adjacent due to wrap-around?

A) Only left and right squares

B) Only top and bottom squares

C) Both left-right AND top-bottom squares are adjacent (wrap-around applies to both rows and columns)

D) No squares wrap around in a four-input K-Map

Correct Answer
Option C

Karnaugh Maps-(Don’t Cares)

MCQs

2. What does an "X" (don't care) in a K-Map or truth table represent?

A) An output that is always 0

B) An output that is always 1

C) An output whose value is unspecified/irrelevant, which can be treated as either 0 or 1, whichever gives the smaller formula

D) An invalid input combination that always causes an error

Correct Answer
Option C

Karnaugh Maps-(Don't Cares)

MCQs

3. In the example of a three-input circuit whose inputs can never all be 1 simultaneously, why do don't-care values help simplify the formula?

A) Because don't cares always force the output to 0

B) Because input combinations guaranteed never to occur can be assigned whichever output value (0 or 1) yields a smaller/simpler formula

C) Because don't cares eliminate the need for a K-Map entirely

D) Because don't cares only apply to four-input functions

Correct Answer
Option B



THANK YOU

Team DLCD

Department of CSE and CSE(AI &ML)



Digital Logic and Computer Design

Unit1
Combinational and Sequential Logic Design
Topic:Combinational Buiding Blocks

Team DLCD
Department of CSE and CSE(AI &ML)

Combinational Building Blocks

OUTLINE

- Combinational logic is often grouped into larger building blocks to build more complex systems.
- This is an application of the principle of abstraction, hiding the unnecessary gate-level details to emphasize the function of the building block.
- Commonly used building blocks: Multiplexers , Decoders

PES UNIVERSITY

Multiplexer (Mux)

Multiplexers are among the most commonly used combinational circuits. A Multiplexer is a digital switch that selects one of several inputs and forwards it to a single output.
They choose an output from among several possible inputs based on the value of a select signal. A multiplexer is sometimes affectionately called a mux.

Multiplexers reduce hardware and simplify circuit design.

Multiplexer (Mux)

- Selects between one of *N* inputs to connect to output
- **$\log_2 N$-bit select input – control input**
- **Example:**

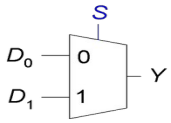
The Boolean expression is:

$$Y = \bar{S}\,D_0 + SD_1$$

2:1 Multiplexer

Symbol and Truth Table

S is called Control Signal/Select input



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S	Y
0	D_0
1	D_1

Combinational Building Blocks

Number of Select Lines

The slide mentions

$$\log_2 N$$

where

- N = number of inputs

This tells us how many **select lines** are required.

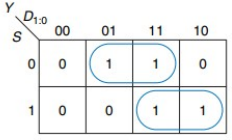
Inputs	Select Lines
2	1
4	2
8	3
16	4

Combinational Building Blocks

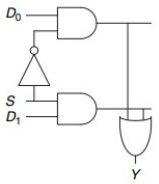
Two different ways to implement a 2:1 Multiplexer:

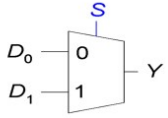
1. Logic gates

- Sum-of-products form



$Y = D_0\bar{S} + D_1 S$

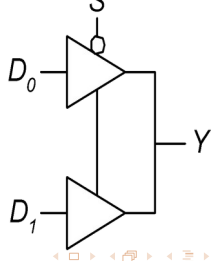




2. Tristates

Multiplexer using tristate buffers

- For an N-input mux, use N tristates
- Turn on exactly one to select the appropriate input



Combinational Building Blocks

A **tri-state buffer** has three output states: **0, 1, and High Impedance (Z)**. In a tri-state MUX, **only one buffer is enabled at a time**, ensuring that only the selected input reaches the output while all others remain disconnected (Z).

How does the Tri-state MUX work?

For a 2:1 MUX:

The select signal controls which buffer is enabled.

If **S = 0**:

D_0 buffer is enabled.

D_1 buffer outputs **Z**.

So, **Y = D_0**.

If **S = 1**:

D_1 buffer is enabled.

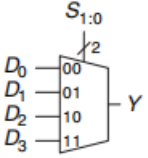
D_0 buffer outputs **Z**.

So, **Y = D_1**.

Only one buffer drives the output at a time.

Combinational Building Blocks

4:1 Mux



S1 S0 Output

------ --------

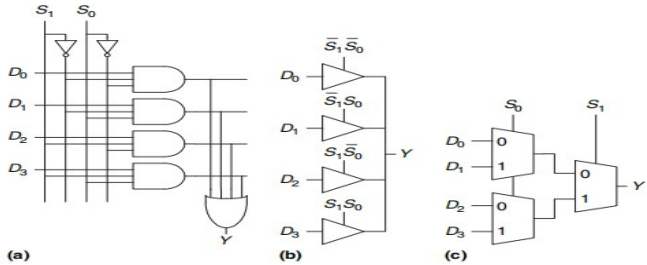
00 → D0

01 → D1

10 → D2

11 → D3

4:1 multiplexer implementations:
(a) two-level logic, (b) tristates, (c) hierarchical



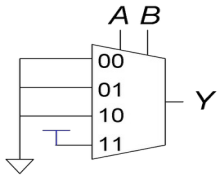
Logic using Multiplexers

Multiplexers can be used as lookup tables to perform **logic functions.**

Using the mux as a lookup table to implement AND gate.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$Y = AB$



4:1 multiplexer implementation of two-input **AND** Function

The inputs, A and B, serve as select lines. The multiplexer data inputs are connected to 0 or 1 according to the corresponding row of the truth table.

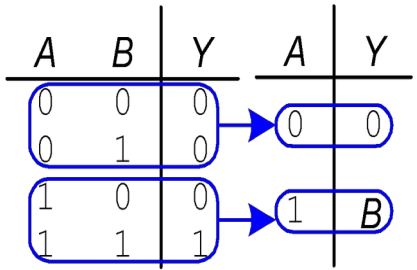
Combinational Building Blocks

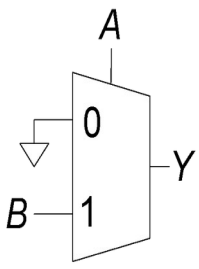
Logic using Multiplexers- (AND implementation)

we can cut the multiplexer size in half, using only a 2^{N-1} -input multiplexer to perform any N-input logic function. The strategy is to provide one of the literals, as well as 0's and 1's, to the multiplexer data inputs.

- Reducing the size of the mux (2:1 Mux)

$Y = AB$

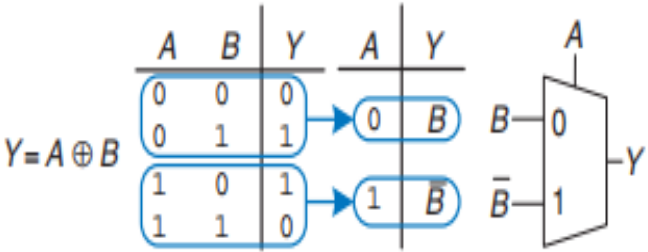




For example, in the case of AND,
when A = 0, Y = 0, regardless of B.
When A = 1, Y = 0 if B = 0 and Y = 1 if B = 1, so Y = B. We then use the multiplexer as a lookup table according to the new, smaller truth table.

Logic using Multiplexers - XOR

- Reducing the size of the mux



Combinational Building Blocks

Logic using Multiplexers - OR ?

Combinational Building Blocks

Logic using Multiplexers - OR ?

A	B	Y(A+B)
0	0	0
0	1	1
1	0	1
1	1	1

Assume:
Select line (S) = A
Data inputs = I_0 and I_1

A	Y(A+B)
0	B-I_0
0	B-I_0
1	1-I_1
1	1-I_1

Look only at the rows where **A = 0**.

A	B	Output
0	0	0
0	1	1

The output follows **B**.
So,
I_0 = B

Look only at the rows where **A = 1**.

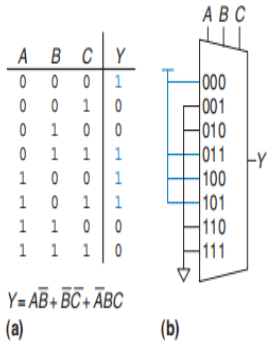
A	B	Output
1	0	1
1	1	1

The output is always **1**.
So,
I_1 = 1

Draw the figure

Logic using Multiplexers

Alyssa P. Hacker needs to implement the function Y = AB'+B'C' + A'BC to finish her senior project, but when she looks in her lab kit, the only part she has left is an 8:1 multiplexer. How does she implement the function?

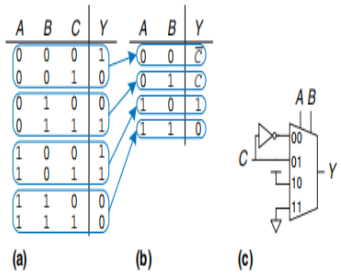


Logic using Multiplexers

Alyssa turns on her circuit one more time before the final presentation and blows up the 8:1 multiplexer. (She accidently powered it with 20 V instead of 5 V after not sleeping all night.) She begs her friends for spare parts and they give her a 4:1 multiplexer and an inverter.

Can she build her circuit with only these parts?

$Y = AB' + B'C' + A'BC$



Implement the function

1. Y=A'B+A'C+AB'

using a **4:1 MUX**.

2. **Implement a 3-variable function using a 4:1 MUX**

Consider:

F(A,B,C)=Σm(1,2,5,7)

Also implement using 2:1 Mux

Combinational Building Blocks

How many 2:1 Mux's are required to implement

1. 4:1
2. 8:1
3. 16:1 and so on

How many 4:1 Mux's are required to implement

4. 8:1
5. 16:1
6. 64:1 and so on

COMBINATIONAL AND SEQUENTIAL LOGIC DESIGN

Combinational Building Blocks

Build an 8:1 MUX
Now we have 8 inputs.
First level
We need:
8→4
So we need **4 × 2:1 MUXes**.
Second level
Those 4 outputs need to become 2 outputs.
So we need **2 × 2:1 MUXes**.
Third level
Finally, 2 outputs need to become 1.
So we need **1 × 2:1 MUX**.
Therefore:
4+2+1=7 8:1=7 two-to-one MUXes

So the general formula is:

2^n:1 MUX requires (2^n–1) 2:1 MUXes

Step 1: First level
A 64:1 MUX has **64 inputs**.
Each 4:1 MUX can reduce **4 inputs to 1 output**.
So:
64/4=16
We need **16 - 4:1 MUXes**.
Step 2: Second level
Now we have 16 outputs.
Each 4:1 MUX can take 4 of these:
16/4=4
So we need **4 - 4:1 MUXes**.
Step 3: Third level
Now we have 4 outputs.
One 4:1 MUX can combine them:
4/4=1
So we need **1 -4:1 MUX**.
Total
16+4+1=21
General formula

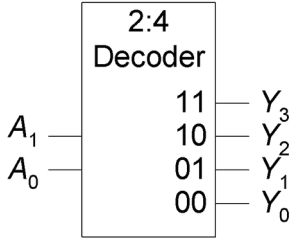
So you can remember the general rule:

$$\text{Number of } k:1 \text{ MUXes to build an } N:1 \text{ MUX} = \frac{N-1}{k-1}$$

Combinational Building Blocks

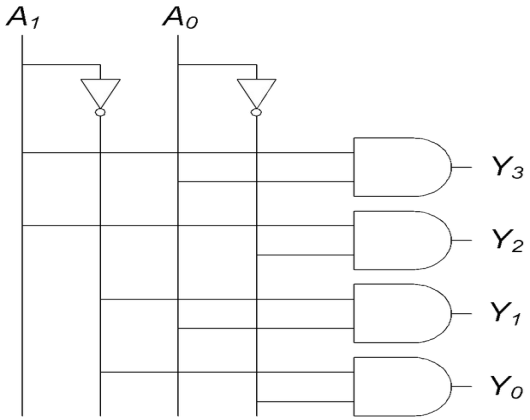
Decoders

- *N* inputs, 2^N outputs
- It asserts exactly one of its outputs depending on the input combination.
- One-hot outputs: only one output HIGH at once



A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

2:4 Decoder Implementation

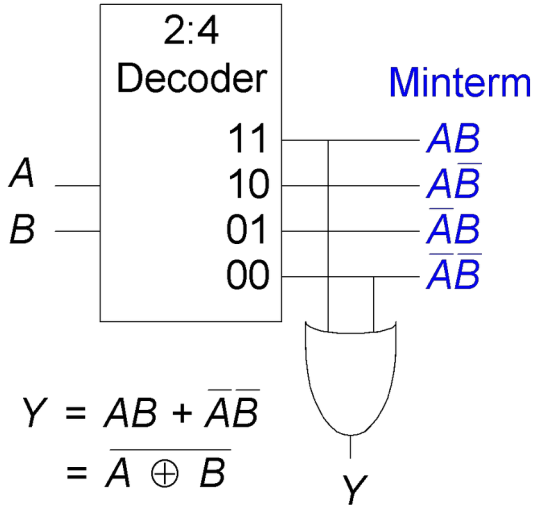


Combinational Building Blocks

Logic Using Decoders

- Decoders can be combined **with OR gates** to build logic functions.

- Ex: Fig shows the two-input XNOR function using a **2:4 decoder and a single OR gate.**



Implementing Logic Functions Using Decoders

- A Boolean function can be represented using a **Truth Table** or **Canonical Sum-of-Products (SOP)** form.
- An **N-input Boolean function** requires an **N : 2^N Decoder**.
  - The decoder generates all possible **minterms**.
  - Each decoder output represents one minterm.
- If the function has **M output 1's** in its truth table:
  - Connect the corresponding **M minterm outputs** of the decoder to an **M-input OR gate**.

General Structure:

N Inputs → N : 2^N Decoder → Selected Minterms → M-input OR Gate → Function Output

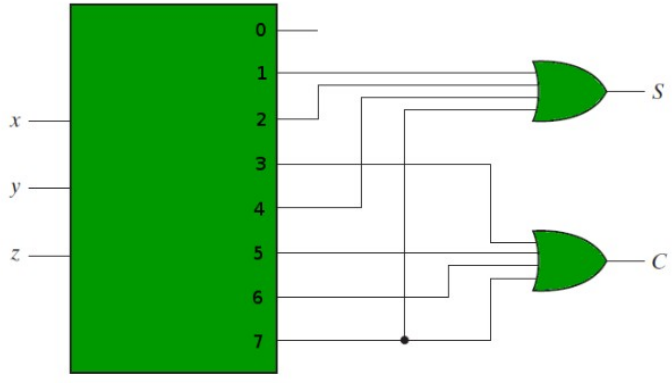
Combinational Building Blocks

Ex: To implement the logic of a full adder, we need a 3:8 decoder and OR gates.

If sum and Carry is

$S=\sum(1,2,4,7)$

$C = \sum(3,5,6,7)$



Encoder

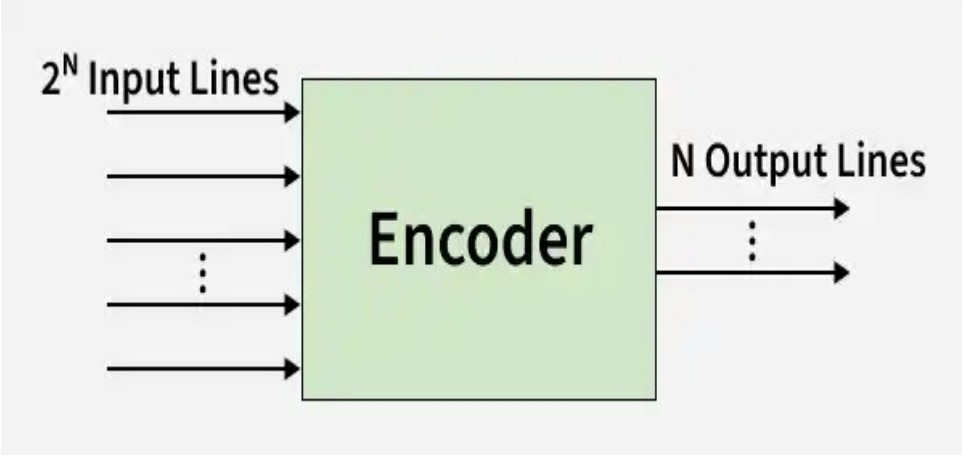
An encoder is a digital combinational circuit that converts multiple input signals into a binary code. It typically has one active input at a time and generates a binary output representing the position of that active input.

The number of inputs is usually 2^n, with n output lines.

- Inputs are usually active high, meaning high voltage represents an active signal.
- Binary-weighted encoders assign codes based on input position using binary values.
- Encoders help convert parallel inputs into compact binary formats for processing.
- Encoders are useful in digital systems for data compression and efficient transmission.
- For example, a 4-to-2 encoder has four inputs and produces a 2-bit output code.

Combinational Building Blocks

The number of inputs is usually 2^n, with n output lines.



PES UNIVERSITY
CELEBRATING 50 YEARS

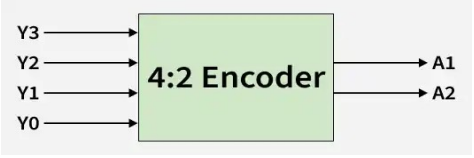
Types of Encoders

1. 4 to 2 Encoder
2. Octal to Binary Encoder (8 to 3 Encoder)
3. Decimal to BCD Encoder
4. Priority Encoder

1. 4 to 2 Encoder

The 4 to 2 Encoder consists of four inputs Y_3, Y_2, Y_1 & Y_0, and two outputs A_1 & A_0. **At any time, only one of these 4 inputs can be '1'** in order to get the respective binary code at the output.

Logic symbol of the 4 to 2 encoder.



Truth Table

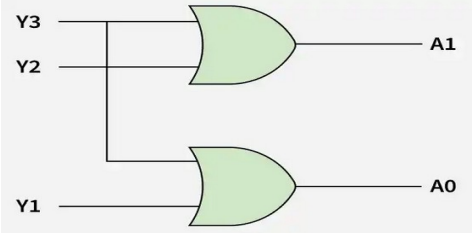
INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0)
0001	00
0010	01
0100	10
1000	11

Logical expression for A_1 and A_0:

$A_1 = Y_3 + Y_2$

$A_0 = Y_3 + Y_1$

Circuit Diagram



2. Octal to Binary Encoder (8 to 3 Encoder)

The 8 to 3 Encoder or octal to Binary encoder consists of 8 inputs: Y_7 to Y_0 and 3 outputs: A_2, A_1 & A_0. Each input line corresponds to each octal digit value and three outputs generate corresponding binary code.

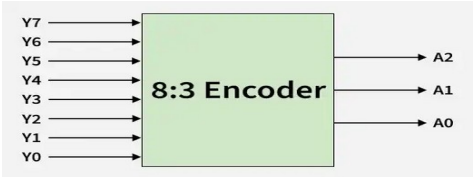
Truth Table

INPUTS($Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$)	OUTPUTS($A_2A_1A_0$)
00000001	000
00000010	001
00000100	010
00001000	011
00010000	100
00100000	101
01000000	110
10000000	111

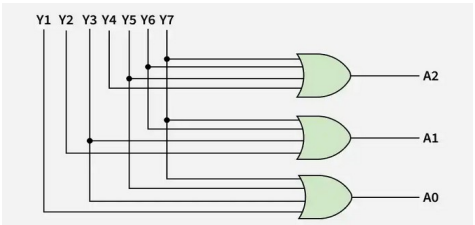
Logical expression for A_2, A_1, and A_0:

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$
$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$
$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

Logic symbol of the Octal or 8 to 3 encoder.



Circuit Diagram



3. Decimal to BCD Encoder

The decimal-to-binary encoder usually consists of 10 input lines and 4 output lines. Each input line corresponds to each decimal digit and 4 outputs correspond to the BCD code.

Truth Table

NPUTS($Y_9Y_8Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$)	OUTPUTS($A_3A_2A_1A_0$)
0000000001	0000
0000000010	0001
0000000100	0010
0000001000	0011
0000010000	0100
0000100000	0101
0001000000	0110
0010000000	0111
0100000000	1000
1000000000	1001

Logical expression for A_3, A_2, A_1, and A_0:

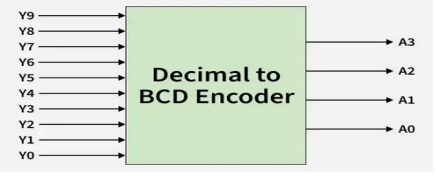
$A_3 = Y_9 + Y_8$

$A_2 = Y_7 + Y_6 + Y_5 + Y_4$

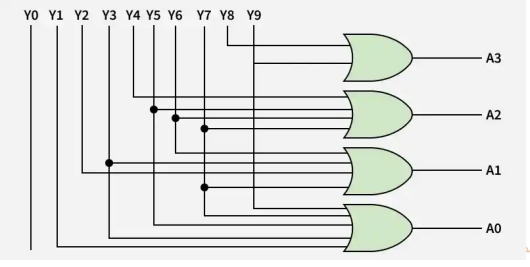
$A_1 = Y_7 + Y_6 + Y_3 + Y_2$

$A_0 = Y_9 + Y_7 + Y_5 + Y_3 + Y_1$

Logic symbol of the BCD encoder.



Circuit Diagram



Combinational Building Blocks

What is an ordinary encoder?

An encoder converts **one active input** into a binary code.
For example, a **4-to-2 encoder** has 4 inputs and 2 outputs:

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0)
0001	00
0010	01
0100	10
1000	11

Here we assume **only one input is 1 at a time**.
For example:
Y3 Y2 Y1 Y0
0 1 0 0 → 10
So, input **Y_2** is active, and the encoder produces binary **10**.

2. What is the problem? (NEED OF PRIORITY ENCODER)

Suppose two inputs become 1:
Y3 Y2 Y1 Y0
0 1 1 0
Which one should the encoder represent?
An ordinary encoder cannot decide because D_2 and D_1 are both active.
This is where the **priority encoder** comes in.

4. Priority Encoder

A 4 to 2 priority encoder **has 4 inputs**: **Y_3, Y_2, Y_1 & Y_0, and 2 outputs: A_1 & A_0.** Here, the input,

Y_3 has the highest priority

Y_0 has the lowest priority.

In this case, even if more than one input is '1' at the same time, the output will be the (binary) code corresponding to the input, which is having higher priority.

Truth Table (Here **X means "don't care"**)

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0V)
0000	XX0
0001	001
001X	011
01XX	101
1XXX	111

Case 1:
Y3 Y2 Y1 Y0
0 0 0 1
Only Y_0 is active.
Output:
A1 A0 = 00

Case 2:
Y3 Y2 Y1 Y0
0 0 1 X
Only Y1 is active.
Output:
A1 A0 = 01

Case 3:
Y3 Y2 Y1 Y0
0 1 X X
Only Y2 is active.
Output:
A1 A0 = 10

Case 4:
Y3 Y2 Y1 Y0
1 X X X
Only Y3 is active.
Output:
A1 A0 = 11

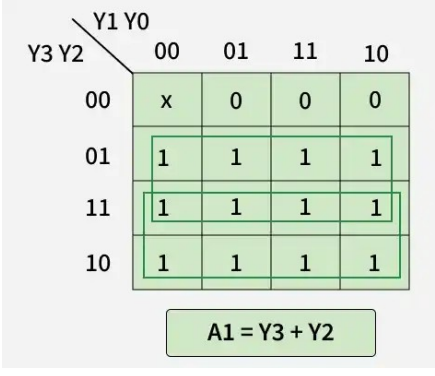
4. Priority Encoder

Truth Table (Here **X means "don't care")**

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0V)
0000	XX0
0001	001
001X	011
01XX	101
1XXX	111

Logical Expressions using K-Map

The logical expression for A_1 is shown below:



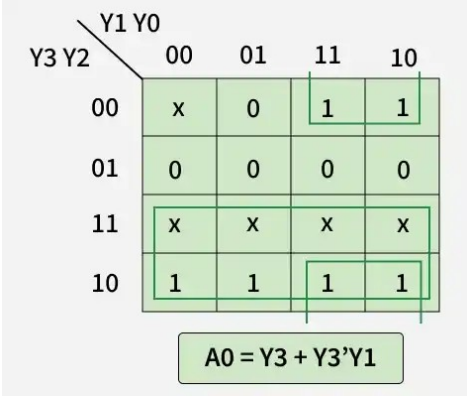
4. Priority Encoder

Truth Table (Here **X means "don't care")**

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0V)
0000	XX0
0001	001
001X	011
01XX	101
1XXX	111

Logical Expressions using K-Map

The logical expression for A_0 is shown below:



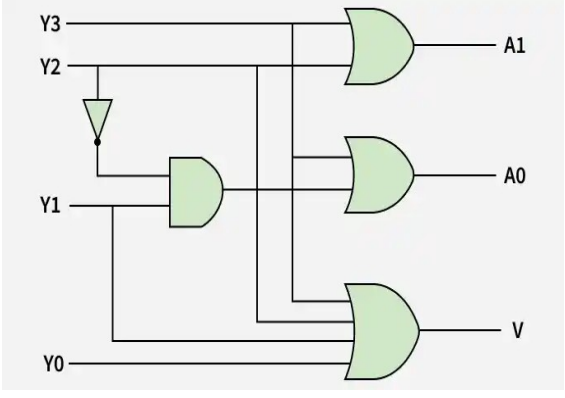
4. Priority Encoder

Truth Table (Here **X means "don't care")**

INPUTS($Y_3Y_2Y_1Y_0$)	OUTPUTS(A_1A_0V)
0000	XX0
0001	001
001X	011
01XX	101
1XXX	111

Why do we need the Valid output(V)?
Just to mention no input is high V=0
All other cases V=1

Circuit Diagram



MUX vs Decoder vs Encoder

Feature	MUX	Decoder	Encoder
Full name	Multiplexer	Decoder	Encoder
Basic function	**Many → One**	**n → 2^n**	**2^n → n**
Inputs	Many data inputs + select lines	n input/select lines	2^n input lines
Outputs	1	2^n	n
Main purpose	Select **one input**	Activate **one output**	Identify **which input is active**
Select lines	Yes	Yes/input lines act as selection code	No separate select lines
Example	4:1 MUX	2:4 Decoder	4:2 Encoder
Typical output	Selected data	One-hot output	Binary code
Multiple active inputs?	Not applicable to data selection	Output depends on input combination	Usually assumes one active input
Special version	—	Priority decoder, etc.	**Priority encoder**

PES UNIVERSITY

MCQ 1

Q1. A 4:1 Multiplexer requires how many select lines?

A) 1
B) 2
C) 3
D) 4

Answer: B) 2
Explanation:
A multiplexer with N inputs requires $\log_2 N$ select lines.
For a 4:1 MUX:

$$\log_2 4 = 2$$

MCQ 2

Q2. A decoder with N inputs has how many outputs?

A) N
B) N^2
C) 2^N
D) $N + 2$

Answer: C) 2^N
Explanation:
A decoder activates exactly one of 2^N outputs depending on the input combination.
Example:
- 2 inputs → 4 outputs
- 3 inputs → 8 outputs
- 4 inputs → 16 outputs

MCQ 3

Q3. In a decoder, the outputs are called one-hot because

A) All outputs are HIGH simultaneously.

B) Exactly one output is HIGH for any input combination.

C) All outputs are LOW.

D) Outputs keep changing randomly.

Answer: B) Exactly one output is HIGH for any input combination.

Explanation:

For every valid input, only one decoder output becomes HIGH while all others remain LOW.



THANK YOU

Team DLCD
Department of CSE and CSE(AI &ML)



Digital Logic and Computer Design

Latches, Flip-flops

Team DLCD
Department of CSE and CSE(AI &ML)

Digital Logic and Computer Design

Latches, Flip-flops

Team DLCD

Department of CSE and AIML

Course Outline

Digital Design

- Combinational logic design
- Sequential logic design
  - **Latches, Flip-flops**

How does Sequential Logic differ from Combinational Logic?

- Previously we analysed and designed combinational logic. The output of combinational logic depends only **on current input values.** Given a specification in the form of a truth table or Boolean equation, we can create an optimized circuit to meet the specification.

- Now we will analyse and design sequential logic. The outputs of sequential logic depend on **both current and prior input values**.

Sequential Circuits

- It consists of a ***combinational circuit*** to which ***storage elements*** are connected to form a ***feedback path.***
- The storage elements are devices capable of storing ***binary information***.
- The binary information stored in these elements at any given time *defines the* ***state*** *of the sequential circuit* at that time.
- The sequential circuit receives binary information from external inputs that, together with the present state of the storage elements, determine the binary value of the outputs.
- These **external inputs** also determine the condition for changing the state in the storage elements.
- The ***next state*** of the *storage elements* is also a function of ***external inputs*** and the ***present state.***

Types of Sequential Circuits

There are two main types of sequential circuits, and their classification is a function of the timing of their signals.

- A **synchronous sequential circuit** is a system whose behavior can be defined from the knowledge of its signals at discrete instants of time.
- The behavior of an **asynchronous sequential circuit** depends upon the **input signals** at any instant of time and the order in which the inputs change.

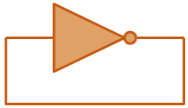
Two Memory Elements

LATCH

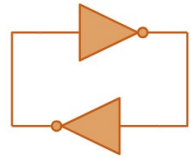
FLIPFLOP

How to Implement Memory?

- The inverter is essentially the simplest logic gate
- What happens when we connect its output to input, forming a loop?
- Inverter loop is thus not in a **stable state**
- Hence, a **single inverter loop cannot store a stable bit**.



- How about a loop of two inverters?
- A two-inverter loop can be in one of **two stable states**.
- A bit can have one of **two different values**.
  Thus, a two-inverter loop can **store** (or "remember") a single bit.
- But how can we **change the bit stored** or the **stable state**?



How to Implement Memory?

How do we CHANGE the stored bit?

This is the crucial problem.
Suppose the circuit currently stores:
A=0, B=1
The feedback keeps reinforcing this state.
How can we force it to become:
A=1, B=0
We need **some external control mechanism** to temporarily override the feedback.

So we need **cross coupled inverters**

Latches and Flip Flops

- The fundamental building block of memory is a **bistable element**, an element with two stable states.

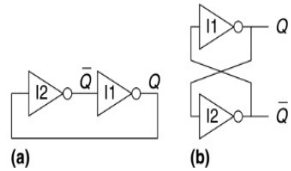
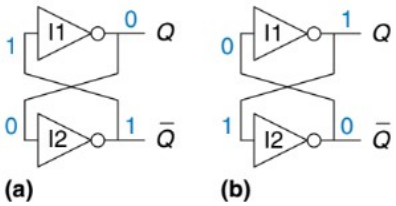


Figure 3.1 Cross-coupled inverter pair

- The inverters are cross-coupled, meaning that the input of I1 is the output of I2 and vice versa.

Latches and Flip Flops

- **Bistable operation of cross-coupled inverters**



The cross-coupled inverters have two stable states, Q = 0 and Q = 1, so the circuit is said to be bistable.

Case I: Q = 0

As shown in Figure (a), I2 receives a FALSE input, Q, so it produces a TRUE output on Q' I1 receives a TRUE input, Q' so it produces a FALSE output on Q. This is consistent with the original assumption that Q = 0, so the case is said to be stable.

Case II: Q = 1

As shown in Figure (b), I2 receives a TRUE input and produces a FALSE output on Q'. I1 receives a FALSE input and produces a TRUE output on Q. This is again stable.

Latches and Flip Flops

Problem with cross coupled inverters

Although the cross-coupled inverters can store a bit of information, they are not practical because the user has no inputs to control the state.

However, other bistable elements, such as latches and flip-flops, provide inputs to control the value of the state variable. The remainder of this section considers these circuits.

Latches and Flip Flops

- Although the cross-coupled inverters can store a bit of information, they are not practical because the **user has no inputs to control the state**.

- However, other bistable elements, such **as latches and flip-flops**, provide inputs to control the value of the state variable.

Latch

A latch is a 1-bit memory element that stores a value and can change its output as long as its enable/control signal is active.

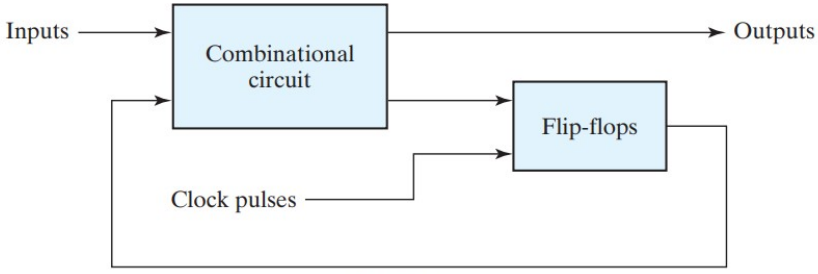
In simple terms:

Latch = level-sensitive memory

For example, an **Enable** signal controls the latch:

- Enable = 1 → latch can change/update
- Enable = 0 → latch holds the previous value

Latches and Flip Flops



(a) Block diagram



(b) Timing diagram of clock pulses

SR Latch

One of the simplest sequential circuits is the SR latch, which is composed of two cross-coupled NOR gates. The SR latch is similar to the cross-coupled inverters, but its state can be controlled through the S and R inputs, **which set and reset the output Q.**

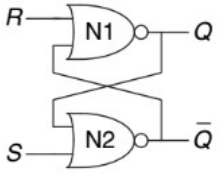


Figure 3.3 SR latch schematic

Case	S	R	Q	$\bar{Q}$
IV	0	0	Q_{prev}	$\bar{Q}_{prev}$
I	0	1	0	1
II	1	0	1	0
III	1	1	0	0

Figure 3.5 SR latch truth table

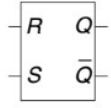


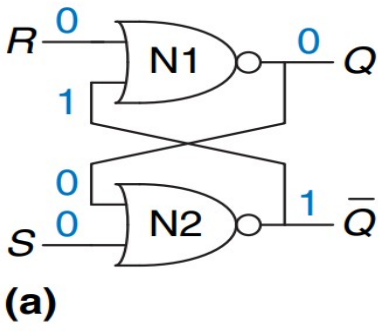
Figure 3.6 SR latch symbol

Cross-coupled inverters + control inputs = SR latch

SR Latch

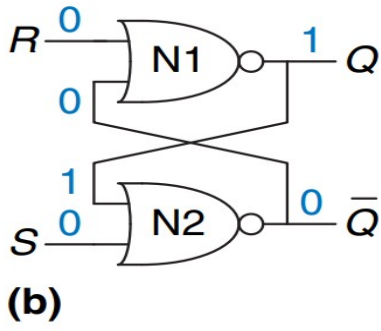
SR Latch Operation when S=R=0

a) Q=0 and Q'=1



(a)

b) Q=1 and Q'=0



(b)

SR Latch

- Like the cross-coupled inverters, the SR latch is a bistable element with one bit of state stored in Q.
- However, the state can be controlled through the S and R inputs. When R is asserted, the state is reset to 0.
- When S is asserted, the state is set to 1. When neither is asserted, the state retains its old value.
- No matter what pattern of setting and resetting occurred in the past, all that is needed to predict the future behavior of the SR latch is whether it was most recently set or reset.
- **S=1 → Q becomes 1**
- **S=0,R=0 → Q stays 1**
- **R=1 → Q becomes 0**
- **S=0,R=0 → Q stays 0**
- **S=1 → Q becomes 1**

Problems with SR Latch

1. Problem with the SR latch

In an SR latch:

- S tells us **what to store** → Q=1
- R tells us **what to store** → Q=0

But they also directly cause the change.

For example:

S=1 ⇒ Q becomes 1

There is no separate signal saying: "Change now."

So **what to store** and **when to change** are mixed together.

2. The problem with both S and R

For the SR latch:

S	R	Action
0	0	Hold
0	1	Reset → (Q=0)
1	0	Set → (Q=1)
1	1	✗ Invalid

The S=R=1 case is awkward.

So we want a simpler memory element where we don't have to separately worry about Set and Reset.

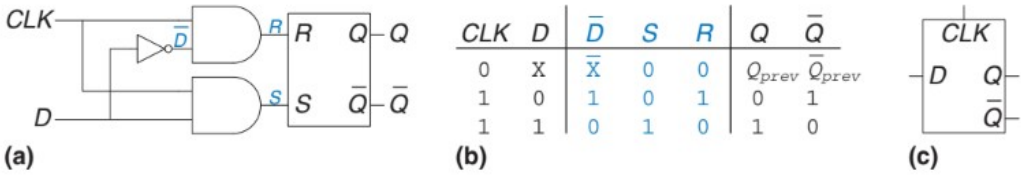
D Latch

- Designing circuits becomes easier when these questions of **what and when are separated**.
- D latch solves these problems.
- It has two inputs. The data input, D, controls what the next state should be. The clock input, CLK, controls when the state should change.

So:
D=WHAT to store

So:
CLK=WHEN to change

D Latch



CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	X	$\overline{X}$	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

Figure 3.7 D latch: (a) schematic, (b) truth table, (c) symbol

- When CLK = 1, the latch is transparent(open). The data at D flows through to Q as if the latch were just a buffer. When CLK = 0, the latch is opaque(closed).
- It blocks the new data from flowing through to Q, and Q retains the old value. Hence, the D latch is some- times called a transparent latch or a level-sensitive latch.
- The circuit guarantees that S and R **can never both be 1**.

Problem with latch (Need of Flipflop)

- The problem with the latch is that it responds to a change in the level of a clock pulse.

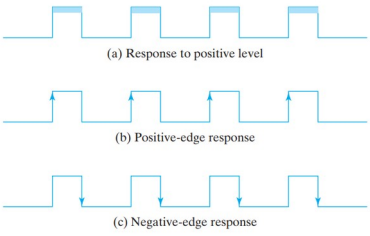


FIGURE 5.8
Clock response in latch and flip-flop

As shown in Fig. 5.8 (a), a positive level response in the enable input allows changes in the output when the D input changes while the clock pulse stays at logic 1. **The result is an unpredictable situation, since the state of the latches may keep changing for as long as the clock pulse stays at the active level.** The key to the proper operation of a flip-flop is to trigger it only during a signal transition .

Problem with latch (Need of Flipflop)

1.Problem with the latch (fig 5.8a)

CLK = 1 D changes:

D: 0 — 1 — 0 — 1

↓ ↓ ↓

Q: 0 — 1 — 0 — 1

While CLK is HIGH, every change in D can potentially change Q.
So if D changes many times while: CLK=1
then Q can also change many times.
This is **unpredictable** because the final state can depend on exactly when the input changes during the active level.

2. What do we really want?
We want the memory element to say:
"I will look at D **only at one particular instant**."
Not:
"I will keep looking at D for the entire HIGH period."
That particular instant is called an **edge**.

3. What is an edge?
A clock signal has two transitions:
Rising edge (fig 5.8b)
0→1
This is also called the **positive edge** or **positive-going edge**.
Falling edge (fig 5.8c)
1→0 This is the **negative edge** or **negative-going edge**.

Flip-flop changes only at the edge
This is the fundamental difference:
Latch
Level sensitive
For positive-level latch:
CLK=1⇒Q may change

Flip-flop
Edge sensitive
For positive-edge flip-flop:
CLK:0→1⇒Q changes
only at that instant.

Difference b/w Latches & Flip-Flops

Latches	Flip-Flops
Latches are building blocks of sequential circuits, these can be built from **logic gates**	Flip flops are also building blocks of sequential circuits. But, these can be built from the **latches**.
Latch continuously **checks its inputs and changes its output** correspondingly.	Flip flop continuously checks its inputs and changes its output correspondingly only at times determined by **clocking signal**
The latch is **sensitive to the duration of the pulse** and can send or receive the data when the switch is on.	Flipflop is sensitive to a **signal change**. They can transfer data only at the single instant and data cannot be changed until next signal change.Flip flops are used as a register.
It is based on the enable function input.	It works on the basis of clock pulses.
It is a level triggered, it means that the output of the present state and input of the next state depends on the level that is binary input 1 or 0. (**asynchronous circuits)**	It is an edge triggered, it means that the output and the next state input changes when there is a change in clock pulse whether it may a+ve or-ve clock pulse.(**synchronous circuits**)

Difference b/w Latches & Flip-Flops

Example: Traffic Light Controller
Scenario:
A city traffic light controller uses digital storage elements to decide when to change from Green → Yellow → Red.

Using a Latch (Level-triggered)
Suppose the system uses a **latch** to store the current state of the traffic light.
Since a latch is **level-sensitive**, whenever the clock/enable signal is **high**, the latch is "transparent" — it immediately passes input changes to the output.

Problem: If there is noise or a small glitch during the enable-high period, the latch will capture it. This could cause the traffic light to change state unexpectedly (e.g., Green jumps to Red without Yellow).

Application-level interpretation: **Not safe** for traffic control, because continuous input changes during the active level may corrupt the output.

Difference b/w Latches & Flip-Flops

Example: Traffic Light Controller
Scenario:
A city traffic light controller uses digital storage elements to decide when to change from Green → Yellow → Red.

Using a Flip-Flop (Edge-triggered)
If the system uses a **Flip-Flop**, the light state is updated **only at the clock's rising (or falling) edge**.
Input changes happening in between edges are ignored until the next edge.
Advantage: The traffic light changes state in a controlled manner (e.g., every 30 seconds on the rising clock edge). Even if input glitches occur while the clock is low, they won't affect the state.

Application-level interpretation: **Much safer and reliable** for real-world sequential control (traffic lights, elevators, etc.).

Difference b/w Latches & Flip-Flops

https://www.youtube.com/watch?v=LTtuYeSmJ2g

What is Flipflop and how to build it?

There are two ways that a latch can be modified to form a flip-flop.

- **One way is to employ two latches in a special configuration that isolates the output of the flip-flop and prevents it from being affected while the input to the flip-flop is changing – Example-D flipflop**
- Another way is to produce a flip-flop that triggers only during a signal transition (from 0 to 1 or from 1 to 0) of the synchronizing signal (clock) and is disabled during the rest of the clock pulse.

D Flip-Flop

- A D flip-flop can be built from two back-to-back D latches controlled by complementary clocks

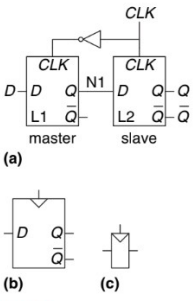


Figure 3.8 D flip-flop: (a) schematic, (b) symbol, (c) condensed symbol

The first latch, L1, is called the master. The second latch, L2, is called the slave. The node between them is named N1.

When CLK = 0, the master latch is transparent and the slave is opaque. Therefore, whatever value was at D propagates through to N1.
When CLK = 1, the master goes opaque and the slave becomes transparent.

D Flip-Flop

- A D flip-flop can be built from two back-to-back D latches controlled by complementary clocks

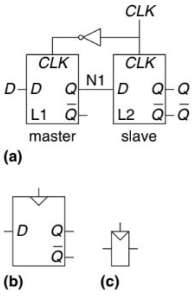


Figure 3.8 D flip-flop: (a) schematic, (b) symbol, (c) condensed symbol

Hence, whatever value was at D immediately before the clock rises from 0 to 1 gets copied to Q immediately after the clock rises.
At all other times, Q retains its old value, because there is always an opaque latch blocking the path between D and Q.

In other words, a D flip-flop copies D to Q on the rising edge of the clock, and remembers its state at all other times.
The rising edge of the clock is often just called the clock edge for brevity. The D input specifies what the new state will be.

D Flip-Flop

- A D flip-flop is also known as a master-slave flip-flop, an edge-triggered flip-flop, or a positive edge-triggered flip-flop. The triangle in the symbols denotes an edge-triggered clock input. The Q output is often omitted when it is not needed.

The main idea is:

When the clock triggers the flip-flop, whatever is present at D gets stored in Q.

So we can remember it as:

Q(next) = D

D	Clock event	Q(next)
0	↑	0
1	↑	1

Here ↑ means the **rising edge** of the clock.

Flip-Flops(2ND METHOD)

Flip-Flops have following representation:

- Function table- output Q and Q`
- Characteristic table- They define next state
- Characteristic equation-solve using K map- The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation.
- Excitation table- predict the i/p based on current state and next state to build state diagram

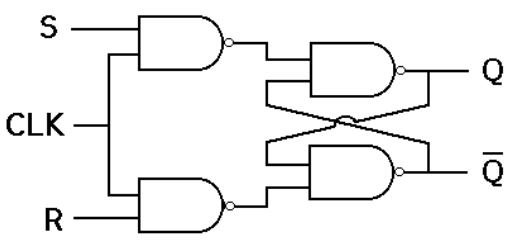
Characteristic table & Excitation Table

The easiest way to remember it is:

Characteristic table = What the flip-flop DOES

Excitation table = What we MUST GIVE to the flip-flop to make it DO what we want

SR Flip-Flops Characteristic table and Characteristic equation



INPUTS			OUTPUT	STATE
CLK	S	R	Q	
X	0	0	No Change	Previous
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	-	Forbidden

Characteristic table

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

Characteristic equation by simplifying using K maps

$Q_{n+1} = S + R`Q_n$

Excitation Table and State Diagram

S	R	Present state Q_n	Next state Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

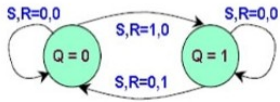
Invalid states

Truth table of SR flip flop

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Excitation table of SR flip flop

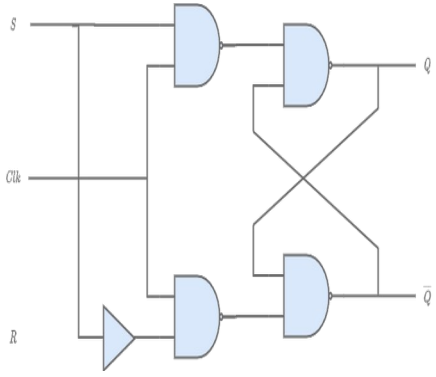
Characteristic Table of SR

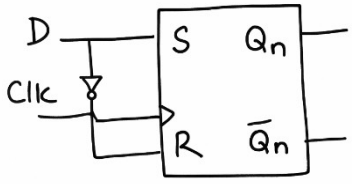


State Diagram

Source: Electrically4u

D Flip-Flop-Data FF





D Flip-Flop

En	D	Next state of Q
0	X	No change
1	0	Q = 0; reset state
1	1	Q = 1; set state

(b) Function table

D Flip-Flop

D	Q(t + 1)	
0	0	Reset
1	1	Set

Characterstic Table

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Simplifying using K maps- characteristic equation is

$$Q(t + 1) = D$$

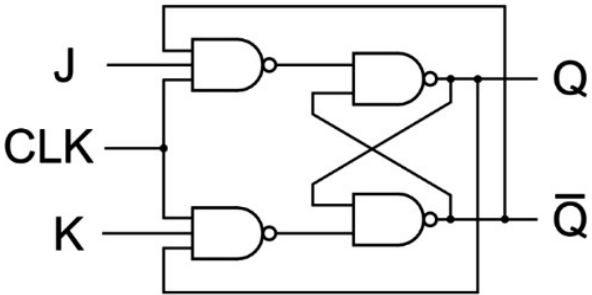
Excitation table

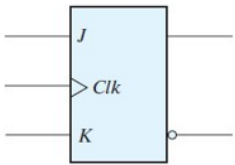
Qn	Q(n+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

JK Flip-Flop

A **JK flip-flop** is required mainly to overcome the limitations of the **SR flip-flop** and to provide a more versatile storage element in sequential circuits.

In an **SR flip-flop**, when **S=1** and **R=1** simultaneously, the output becomes indeterminate (invalid).





(b) Graphic symbol

JK Flip-Flop-function table/Truth table

Clk	J	K	Q
0	x	x	Qn
1	0	0	Qn
1	0	1	0
1	1	0	1
1	1	1	$\overline{Qn}$



Memory

Toggle

race around condition

JK Flip-Flop

Truth Table

Clk	J	K	Q
0	x	x	Qn
1	0	0	Qn
1	0	1	0
1	1	0	1
1	1	1	Qn

Characteristics table

Q(n)	J	K	Q(n+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Excitation table

Q(n)	Q(n+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

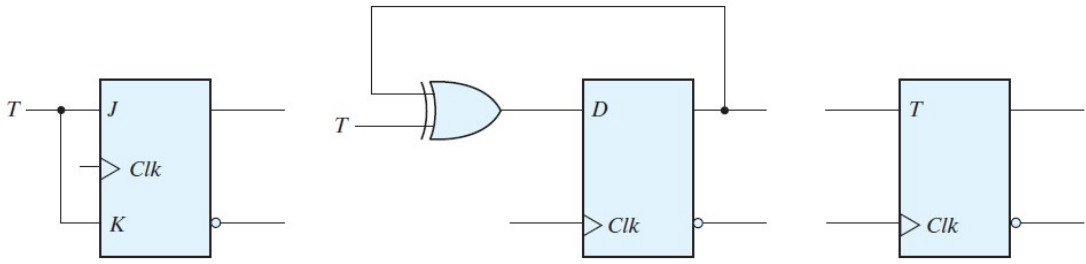
Solve using K map:

$J = Q_{n+1}$ $K = Q'_{n+1}$

Solve using K map to get characterstic equation:

$Q_{n+1} = Q_nK` + Q_n`J$

T Flip-Flop (Toggle Flip-Flop)



(a) From *JK* flip-flop (b) From *D* flip-flop (c) Graphic symbol

T Flip-Flop (Toggle Flip-Flop)

Truth table- if clk=0 then previous state

CLK	T	Q_{n+1}
↑	0	Q_n
↑	1	Q_n'

Characterstic Table

T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

Excitation table

Qn	Qn+1	T
0	0	0
0	1	1
1	0	1
1	1	0

$$Q(t+1) = T \oplus Q = TQ' + T'Q$$

Flip-Flops

- JK flip-flop can function as:
- **SR flip-flop** (J = S, K = R)
- **T flip-flop** (J=K=1 → toggles)
- **D flip-flop** (J=D, K=¬D)
- Hence, it's a **universal flip-flop** that can emulate other types.

Characteristics Table -summary

Flip-Flop Characteristic Tables

JK Flip-Flop

J	K	Q(t + 1)	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

D Flip-Flop

D	Q(t + 1)	
0	0	Reset
1	1	Set

T Flip-Flop

T	Q(t + 1)	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

Characteristics Equations-summary

The logical properties of a flip-flop, as described in the characteristic table, can be expressed algebraically with a characteristic equation. For the D flip-flop, we have the characteristic equation

$$Q(t + 1) = D$$

which states that the next state of the output will be equal to the value of input D in the present state. The characteristic equation for the JK flip-flop can be derived from the characteristic table or from the circuit of Fig. 5.12. We obtain

$$Q(t + 1) = JQ' + K'Q$$

where Q is the value of the flip-flop output prior to the application of a clock edge. The characteristic equation for the T flip-flop is obtained from the circuit of Fig. 5.13:

$$Q(t + 1) = T \oplus Q = TQ' + T'Q$$

Steps for Flipflop Conversion



JK to D FF (b)

- STEP1-Identify available and required FF
  Available is JK and required is D

- Step-2 Make characteristic table of required FF

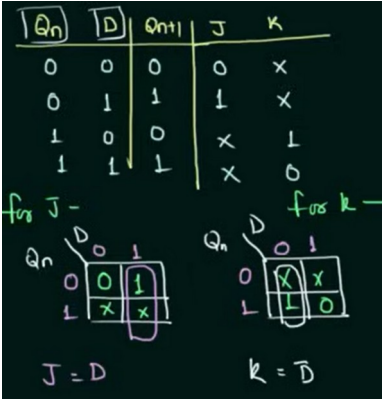
Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

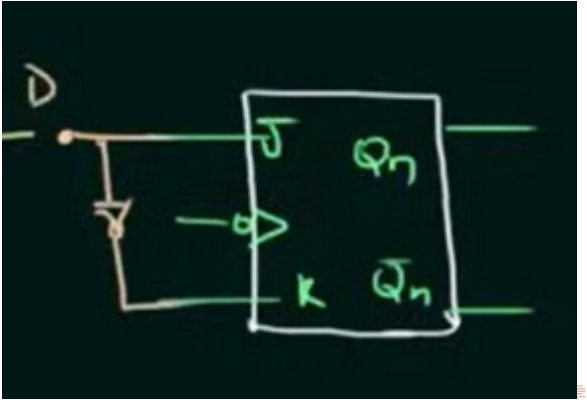
- Step-3- Make Excitation table of available FF

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

JK to D FF

- STEP-4-write Boolean Expression for Available FF(JK) using required FF (D)
- STEP-5 Write the circuit diagram





2. D to JK FF (a)

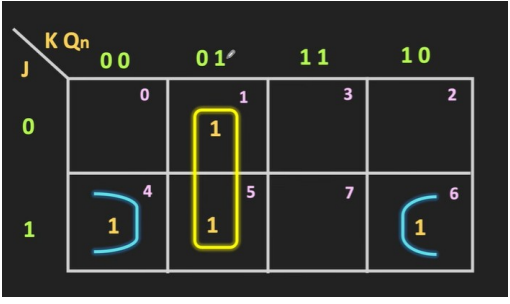
J	K	Q_n	Q_{n+1}	D
0	0	0	0	
0	0	1	1	
0	1	0	0	
0	1	1	0	
1	0	0	1	
1	0	1	1	
1	1	0	1	
1	1	1	0	

Excitation Table of the D Flip-Flop

Q_n	Q_{n+1}	D
0 →	0	0
0 →	1	1
1 →	0	0
1 →	1	1

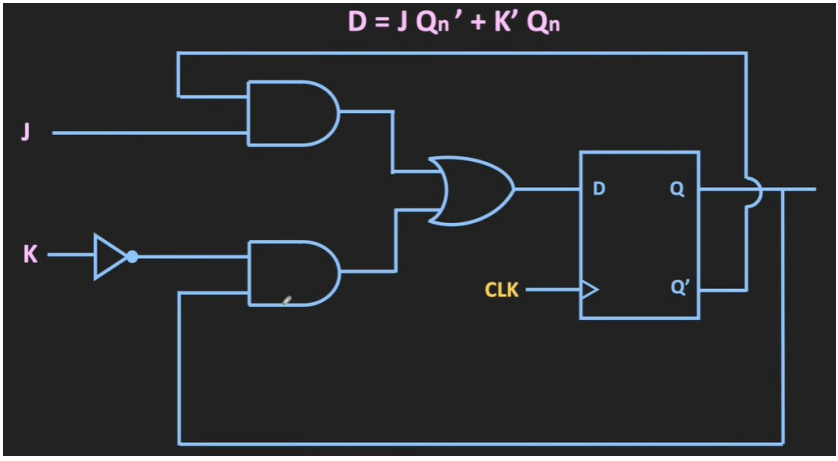
D to JK FF

J	K	Q_n	Q_{n+1}	D
0	0	0 →	0	0
0	0	1 →	1	1
0	1	0 →	0	0
0	1	1 →	0	0
1	0	0 →	1	1
1	0	1 →	1	1
1	1	0 →	1	1
1	1	1 →	0	0



$$D = J Q_n' + K' Q_n$$

D to JK FF



Exercise 3.10

Exercise 3.10 *A JK flip-flop* receives a clock and two inputs, J and K. On the rising edge of the clock, it updates the output, Q. If J and K are both 0, Q retains its old value. If only J is 1, Q becomes 1. If only K is 1, Q becomes 0. If both J and K are 1, Q becomes the opposite of its present state.

(a) Construct a JK flip-flop using a D flip-flop and some combinational logic.

(b) Construct a D flip-flop using a JK flip-flop and some combinational logic.

(c) Construct a T flip-flop (see Exercise 3.9) using a JK flip-flop.

Solve it.

(c) Construct a T flip-flop using a JK flip-flop

Registers

An N-bit register is a bank of N flip-flops that share a common CLK input, so that all bits of the register are updated at the same time.

Registers are the key building block of most sequential circuits.

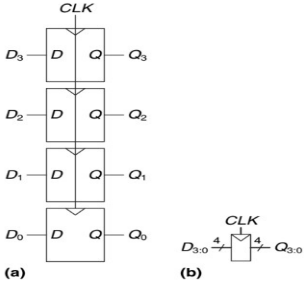


Figure A 4-bit register:
(a) schematic and (b) symbol

Four-bit register with inputs D3:0 and outputs Q3:0. D3:0 and Q3:0 are both 4-bit buses.

Registers

How much information can a N-bit register store?

Each flip-flop stores either **0 or 1**.

Four flip-flops can therefore store:

2^4=16 different combinations.

From:

0000

to:

1111

So a 4-bit register can store **16 different values**.

Hence N bit register = N Flip flops and can store 2^N different combinations.

"N-bit register stores N bits, but it can represent 2^N different values."

Enabled Flip-Flop

- An enabled flip-flop adds another input called EN or ENABLE to determine whether data is loaded on the clock edge.
- When EN is TRUE, the enabled flip-flop behaves like an ordinary D flip-flop.
- When EN is FALSE, the enabled flip-flop ignores the clock and retains its state
- Enabled flip-flops are useful when we wish to load a new value into a flip-flop only some of the time, rather than on every clock edge.

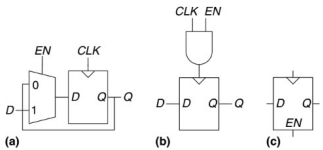


Figure 3.10 Enabled flip-flop: (a, b) schematics, (c) symbol

Enabled Flip-Flop

- In Figure 3.10(a), an input multiplexer chooses whether to pass the value at D, if EN is TRUE, or to recycle the old state from Q, if EN is FALSE.
- In Figure 3.10(b), the clock is gated.
- If EN is TRUE, the CLK input to the flip-flop toggles normally.
- If EN is FALSE, the CLK input is also FALSE and the flip-flop retains its old value.
- EN must not change while CLK = 1, let the flip-flop see a clock glitch(switch at an incorrect time).

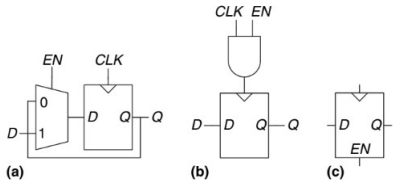


Figure 3.10 Enabled flip-flop: (a, b) schematics, (c) symbol

Enabled Flip-Flop

EN = 1 → LOAD

When:
EN=1
the flip-flop behaves like a normal D flip-flop.
At the clock edge:

$$Q_{next}=D$$

Example
Suppose:
Q = 0
D = 1
EN = 1
When the clock edge arrives:
The new data **1 is loaded**
So output Q=1

EN = 0 → HOLD

Now suppose:
Q = 1
D = 0
but:
EN = 0
When the clock edge arrives, the flip-flop says:
"EN is 0. I am not allowed to load D."
Therefore:

$$Q_{next}=Q$$

So Q remains:
Q=1 Even though D is 0.
i.e. (the new data D is not loaded to Q)

An enabled flip-flop loads new data only when EN = 1; when EN = 0, it ignores the clock edge and keeps its previous value.

Resettable Flip-Flop

- A resettable flip-flop adds another input called RESET. When RESET is FALSE, the resettable flip-flop behaves like an ordinary D flip-flop.
- When RESET is TRUE, the resettable flip-flop ignores D and resets the output to 0.
- Resettable flip-flops are useful when we want to force a known state (i.e., 0) into all the flip-flops in a system when we first turn it on.
- Such flip-flops may be synchronously or asynchronously resettable.
- Synchronously resettable flip-flops reset themselves only on the rising edge of CLK. Asynchronously resettable flip-flops reset themselves as soon as RESET becomes TRUE, independent of CLK.

Resettable Flip-Flop

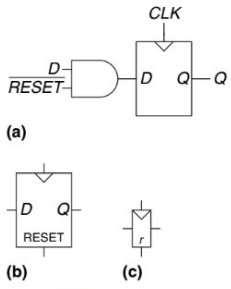


Figure 3.11 Synchronously resettable flip-flop: (a) schematic, (b, c) symbols

- Figure 3.11(a) shows how to construct a synchronously resettable flip-flop from an ordinary D flip-flop and an AND gate.
- When RESET` is FALSE, the AND gate forces a 0 into the input of the flip-flop. When RESET` is TRUE, the AND gate passes D to the flip-flop.
- RESET` is an active low signal
- Asynchronously resettable flip-flops require modifying the internal structure of the flip-flop settable flip-flops are also occasionally used.
- They load a 1 into the flip-flop when SET is asserted
- Resettable and settable flip-flops may also have an enable input and may be grouped into N-bit registers.

LATCHES, FLIP-FLOPS

Resettable Flip-Flop

Normal D Flip-Flop

A normal D flip-flop does:

$Q_{next}=D$

at the active clock edge.

For example:

D = 1 → clock edge → Q = 1

Add RESET

Now we add another input:

D + CLK + RESET

RESET has higher priority than D.

Think of it like this:

IF RESET = 0 → behave normally

IF RESET = 1 → forget D and make Q = 0

So the basic behavior is:

RESET	Clock edge	D	Q(next)
0	↑	0	0
0	↑	1	1
1	↑	X	0

Here **X means "don't care."**

When RESET = 1, we don't care whether D is 0 or 1.

Flip Flop and Latch Comparison

- Latches and flip-flops are the fundamental building blocks of sequential circuits.
- Remember that a D latch is level-sensitive, whereas a D flip-flop is edge-triggered.
- The D latch is transparent when CLK = 1, allowing the input D to flow through to the output Q.
- The D flip-flop copies D to Q on the rising edge of CLK. At all other times, latches and flip-flops retain their old state.
- A register is a bank of several D flip-flops that share a common CLK signal.

Flip-Flop and Latch Comparison

- Ben Bitdiddle applies the D and CLK inputs shown in Figure 3.14 to a D latch and a D flip-flop. Help him determine the output, Q, of each device.

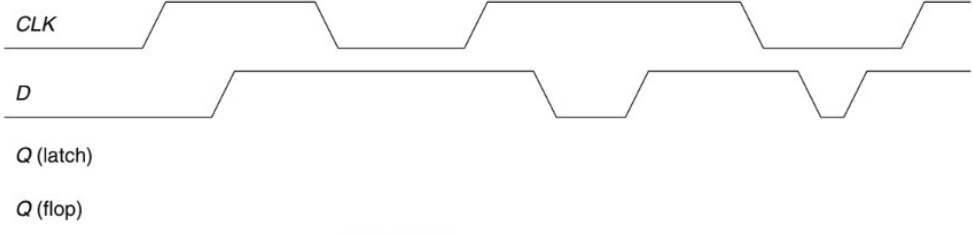


Figure 3.14 Example waveforms

Flip-Flop and Latch Comparison

- Ben Bitdiddle applies the D and CLK inputs shown in Figure 3.14 to a D latch and a D flip-flop. Help him determine the output, Q, of each device.

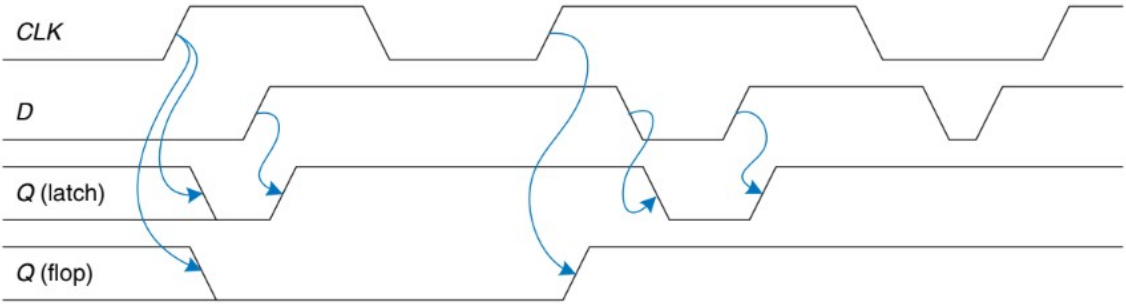


Figure 3.15 Solution waveforms

MCQs

Q1: Why can a loop of two cross-coupled inverters store one bit of information?

A) Because it has two outputs

B) Because it consumes low power

C) Because it can settle into one of two stable states, matching the two values a bit can take

D) Because it uses NAND gates only

LATCHES, FLIP-FLOPS

MCQs

Q1: Why can a loop of two cross-coupled inverters store one bit of information?

A) Because it has two outputs

B) Because it consumes low power

C) Because it can settle into one of two stable states, matching the two values a bit can take

D) Because it uses NAND gates only

MCQs

Q2: In a NOR-gate SR latch, what happens when both inputs S = 1 and R = 1?

A) The latch remembers the previous state

B) The output q is set to 1

C) Both outputs q and q' become 0 — an invalid state since they're no longer complementary

D) The output is reset to q = 0

MCQs

Q2: In a NOR-gate SR latch, what happens when both inputs S = 1 and R = 1?

A) The latch remembers the previous state

B) The output q is set to 1

C) Both outputs q and q' become 0 — an invalid state since they're no longer complementary

D) The output is reset to q = 0

MCQs

Q3: Why is the NAND-gate based SR latch sometimes called an S'R' latch?

A) Because it uses only NOR gates internally

B) Because its input signals are complemented (active-low) compared to the NOR latch

C) Because it stores complemented data only

D) Because S' and R' represent set and reset simultaneously

MCQs

Q3: Why is the NAND-gate based SR latch sometimes called an S'R' latch?

A) Because it uses only NOR gates internally

B) Because its input signals are complemented (active-low) compared to the NOR latch

C) Because it stores complemented data only

D) Because S' and R' represent set and reset simultaneously



THANK YOU

Team DLCD

Department of CSE and CSE(AI &ML)



DIGITAL LOGIC AND COMPUTER DESIGN

Synchronous Sequential Logic Circuits

Team DLCD
Department of CSE and CSE(AI &ML)

DIGITAL LOGIC AND COMPUTER DESIGN

Synchronous Sequential Logic Circuits

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Course Outline

Digital Design

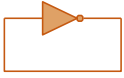
- Combinational logic design
- Sequential logic design
  - **Synchronous Sequential Logic Circuits**

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Potential Problems with Sequential Logic Circuits

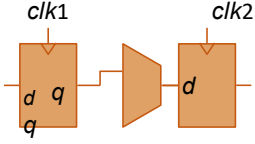
Combinational logic circuits do not contain **loops** or **cyclic paths** but sequential logic circuits do

Loops can cause problems



- Output is unstable

Memory elements can suffer from **race** conditions

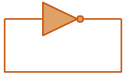


- Output depends upon input sequence

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Sequential Logic Circuits

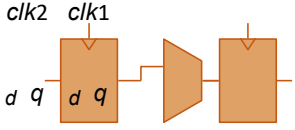
Cyclic paths are broken by inserting flip-flops in them



Cyclic paths can be broken by converting combinational ckts and register(synchronize with clock)

Flip-flops are synchronized by connecting them to a common clock

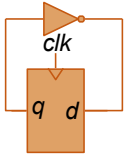
Race conditions are eliminated if the clock time period is long enough



SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

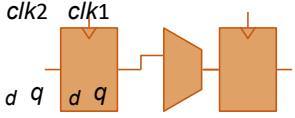
Sequential Logic Circuits

Cyclic paths are broken by inserting flip-flops in them



Flip-flops are synchronized by connecting them to a common clock

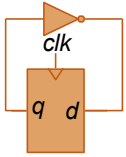
- Race conditions are eliminated if the clock time period is long enough



SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

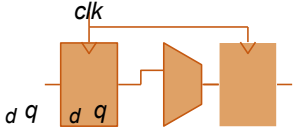
Sequential Logic Circuits

Cyclic paths are broken by inserting flip-flops in them



Flip-flops are synchronized by connecting them to a common clock

- Race conditions are eliminated if the clock time period is long enough



SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Potential Problems with Sequential Logic Circuits

ASTABLE CIRCUITS

Alyssa P. Hacker encounters three misbegotten inverters who have tied themselves in a loop, as shown in Figure 3.16. The output of the third inverter is fed back to the first inverter. Each inverter has a propagation delay of 1 ns. Determine what the circuit does.

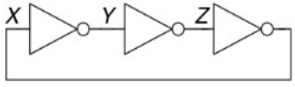


Figure 3.16 Three-inverter loop

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Potential Problems with Sequential Logic Circuits

- Suppose node X is initially 0. Then Y = 1, Z = 0, and hence X = 1, which is inconsistent with our original assumption. The circuit has no stable states and is said to be unstable or astable.
- Figure 3.17 shows the behavior of the circuit. If X rises at time 0, Y will fall at 1 ns, Z will rise at 2 ns, and X will fall again at 3 ns.
- In turn, Y will rise at 4 ns, Z will fall at 5 ns, and X will rise again at 6 ns, and then the pattern will repeat.
- Each node oscillates between 0 and 1 with a period (repetition time) of 6 ns. This circuit is called a ring oscillator.

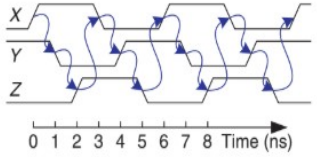


Figure 3.16 Three-inverter loop

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Potential Problems with Sequential Logic Circuits

RACE CONDITIONS

Ben Bitdiddle designed a new D latch that he claims is better than the one because it uses fewer gates. He has written the truth table to find the output, Q, given the two inputs, D and CLK, and the old state of the latch, Qprev.

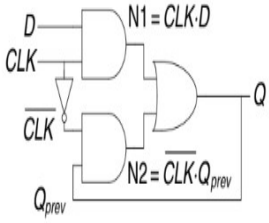
Based on this truth table, he has derived Boolean equations.

He obtains Qprev by feeding back the output, Q in his design.

Does his latch work correctly,independent of the delays of each gate?

CLK	D	Q_{prev}	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = CLK \cdot D + \overline{CLK} \cdot Q_{prev}$$

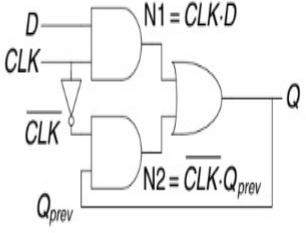


SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Potential Problems with Sequential Logic Circuits

CLK	D	Q_{prev}	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = CLK \cdot D + \overline{CLK} \cdot Q_{prev}$$



- This is an example of asynchronous circuit design in which outputs are directly fed back to inputs.
- Asynchronous circuits are infamous for having race conditions where the behaviour of the circuit depends on which of two paths through logic gates is fastest.

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Problem with Cyclic Paths

- Sequential circuits contain **feedback (cyclic) paths**.
- These paths can cause **race conditions** and **unstable behavior**.
- Analyzing such circuits is difficult and may lead to design errors.

Solution

- Insert **registers** in the feedback path.
- Registers store the system **state** and update only on the **clock edge**.
- This converts the design into **combinational logic + registers**, eliminating races.

Synchronous Sequential Circuit

➢ State changes only at the **rising edge of the clock**.

Uses:

- **Current State** – Present state of the system.
- **Next State** – State after the next clock edge.

- If the clock is slow enough for signals to settle, the circuit operates reliably.

Registers synchronize state changes with the clock, preventing race conditions and ensuring reliable sequential circuit operation.

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Rules of Synchronous Sequential Circuit Composition

A circuit is synchronous if:

- Every circuit element is either a Register or Combinational Logic.
- The circuit contains at least one register.
- All registers share the same clock signal.
- Every feedback (cyclic) path contains at least one register.

Asynchronous Sequential Circuit

- Sequential circuits that do not follow these rules are called Asynchronous Circuits.
- They are more prone to race conditions and unpredictable behavior.

Flip-Flop: Basic Synchronous Sequential Circuit

A Flip-Flop is the simplest synchronous sequential circuit.

Inputs: D (Data), CLK (Clock)

Output: Q

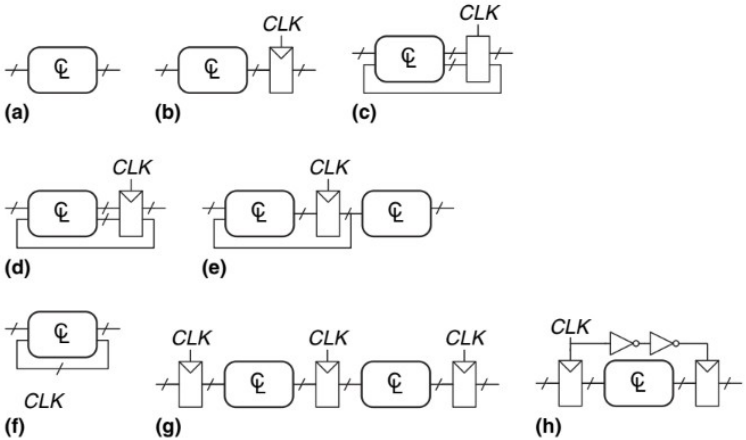
States: {0, 1}

Function: On every clock edge, the next state = D, and Q represents the current state.

✅ **Synchronous circuits use registers and a common clock to ensure predictable, stable, and reliable operation.**

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Rules of Synchronous Sequential Circuit Composition



[1]From "Digital Design and Computer Architecture 2nd Ed.", *Harris and Harris*, ©Elsevier, 2013.

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Rules of Synchronous Sequential Circuit Composition

Circuit (a) is combinational, not sequential, because it has no registers.

(b) is a simple sequential circuit with no feedback.

(c) is neither a combinational

circuit nor a synchronous sequential circuit, because it has a latch that is neither

a register nor a combinational circuit.

(d) and (e) are synchronous sequential logic;

(f) is neither combinational nor synchronous sequential, because it has a cyclic

path from the output of the combinational logic back to the input of the same

logic but no register in the path.

(g) is synchronous sequential logic in the form of a pipeline

(h) is not, strictly speaking, a synchronous sequential circuit, because the second register receives a different clock signal than the first, delayed by two inverter delays.

[1]From "Digital Design and Computer Architecture 2nd Ed.", *Harris and Harris*, ©Elsevier, 2013.

Asynchronous vs. Synchronous Circuit Design

Asynchronous vs. Synchronous Circuit Design

- Asynchronous circuits are more general and flexible since they are not restricted by a global clock.
- Similar to analog circuits, they can use arbitrary feedback and timing.
- Synchronous circuits are easier to design, verify, and implement, making them the preferred choice in digital systems.
- Asynchronous circuits are mainly used for:
- Communication between different clock domains.
- Handling inputs arriving at unpredictable times.
- Research in asynchronous design continues to provide innovative techniques that also benefit synchronous systems.
- Despite decades of research, nearly all modern digital systems are predominantly synchronous due to their simplicity and reliability.

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

Think About It

Classify the following logic circuits as combinational or synchrnous sequential or neither

- AND gate (output not connected to any input)
- AND gate (output connected to an input)
- D flip-flop
- SR latch
- Series of D flip-flops (with common clock) with output of one connected to the input of the next via an inverter

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

MCQs

1. In a sequential logic circuit with an unresolved race condition, why is the output described as "unstable"?

A) Because the circuit consumes too much power

B) Because the flip-flops are not connected to any clock at all

C) Because the output depends on the order/timing of inputs rather than settling to one well-defined value

D) Because combinational gates need more chip area

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

MCQs

1. In a sequential logic circuit with an unresolved race condition, why is the output described as "unstable"?

A) Because the circuit consumes too much power

B) Because the flip-flops are not connected to any clock at all

C) Because the output depends on the order/timing of inputs rather than settling to one well-defined value

D) Because combinational gates need more chip area

MCQs

2. Race conditions are eliminated when flip-flops are synchronized to a common clock, provided that:

A) The clock is asynchronous
B) There is no clock signal
C) Multiple independent clocks are used
D) The clock time period is long enough

MCQs

2. Race conditions are eliminated when flip-flops are synchronized to a common clock, provided that:

A) The clock is asynchronous
B) There is no clock signal
C) Multiple independent clocks are used
D) The clock time period is long enough

SYNCHRONOUS SEQUENTIAL LOGIC CIRCUITS

MCQs

3. An AND gate whose output is fed back into one of its own inputs, with no flip-flop breaking the loop, would be classified as:

A) Synchronous sequential logic
B) Combinational logic
C) Neither combinational nor synchronous sequential (unbroken feedback loop, no clock)
D) A flip-flop

MCQs

3. An AND gate whose output is fed back into one of its own inputs, with no flip-flop breaking the loop, would be classified as:

A) Synchronous sequential logic
B) Combinational logic
C) Neither combinational nor synchronous sequential (unbroken feedback loop, no clock)
D) A flip-flop



THANK YOU

Team DLCD

Department of CSE and CSE(AI &ML)